Part VI

Iterative Methods

Lecture 32. Overview of Iterative Methods

With this lecture the flavor of the book changes. We move from direct methods, a classical topic that is rather thoroughly understood, to the relatively untamed territory of iterative methods. These are the methods that seem likely to dominate the large-scale computations of the future.

Why Iterate?

The importance of iterative algorithms in linear algebra stems from a simple fact: noniterative or "direct" algorithms for general matrices require $O(m^3)$ work. This is too much! It is too much both in the absolute sense that m^3 is huge when m is large, and in the relative sense that since the input to most matrix problems involves only $O(m^2)$ numbers, it seems unreasonable that $O(m^3)$ work must be expended in solving them.

The following table gives a thumbnail history of matrix computations over the years:

1950: $m = 20$	(Wilkinson)
1965: $m = 200$	(Forsythe and Moler)
1980: $m = 2000$	(LINPACK)
1995: $m = 20000$	(LAPACK)

These numbers represent a rough approximation to what dimensions might have been considered "very large" for a dense, direct matrix computation at the indicated dates. In the "Forsythe and Moler era" of the mid-1960s (named here after an influential textbook published in 1967), for example, a matrix of

dimension in the hundreds was large, stretching the limits of what could be calculated on available machines in a reasonable amount of time.

Evidently, in the course of forty-five years, the dimensions of tractable matrix problems have increased by a factor of 10^3. This progress is impressive, but it pales beside the progress achieved by computer hardware in the same period—a speedup by a factor of 10^9, from flops to gigaflops. In the fact that 10^9 is the cube of 10^3, we see played out in history the $O(m^3)$ bottleneck of direct matrix algorithms.

To put it another way, if matrix problems could be solved in $O(m^2)$ instead of $O(m^3)$ operations, some of the matrices being treated today might be 10 to 100 times larger. This is the aim, achieved for some matrices but not others, of matrix iterative methods.

Structure, Sparsity, and Black Boxes

Of course, it is not at all obvious that the $O(m^3)$ bottleneck can be beaten, and indeed, for "random" matrix problems, very likely it cannot. However, the large matrix problems that arise in practice are far from random, and there is a simple reason for this. Small matrices, say with dimension 3 or 30, may arise directly with more or less arbitrary entries in scientific problems—as representations of the relations between three forces in a structure, perhaps, or between thirty species in a chemical reaction. Large matrices, by contrast, usually arise indirectly in the discretization of differential or integral equations. One might say that if m is very large, it is probably an approximation to ∞. It follows that most large matrices of computational interest are simpler than their vast numbers of individual entries might suggest. They have some kind of structure, and as the years have gone by, ways have been found to exploit this structure in more and more contexts.

The most obvious structure that a large matrix may have is *sparsity*, i.e., preponderance of zero entries. (The opposite of *sparse* is *dense*.) For example, a finite difference discretization of a partial differential equation may lead to a matrix of dimension $m = 10^5$ with only $\nu = 10$ nonzero entries per row. This kind of structure is readily exploited by the iterative methods we shall discuss, for these algorithms use a matrix in the form of a *black box*:

$$x \longrightarrow \boxed{\begin{array}{c}\text{BLACK}\\ \text{BOX}\end{array}} \longrightarrow Ax.$$

The iterative algorithm requires nothing more than the ability to determine Ax for any x, which in a computer program will be effected by a procedure whose internal workings need be of no concern to the designer of the iterative algorithm. (Some iterative algorithms also require the computation of A^*x.) For the example of a sparse matrix A, it is easy to design a procedure to compute Ax in only $O(\nu m)$ rather than $O(m^2)$ operations. This is in marked

contrast to the algorithms of direct linear algebra, such as Gaussian or Householder triangularization, which explicitly manipulate matrix entries so as to introduce zeros, but in the process generally destroy sparsity.

Historically, sparsity has been the kind of structure exploited most often in iterative matrix computations. (Sparsity is also exploited by fast direct methods such as nested dissection or minimal degree reordering, not discussed in this book.) More recently, it has become apparent that other kinds of matrix structure may also be exploitable, even though the matrices involved are dense. For example, the solution of integral equations by numerical methods typically leads to dense matrix problems; in engineering these are called boundary element or panel methods. The coefficients of such matrices often have a great deal of regularity in them, and the quest for ways to take advantage of this regularity, for example by multipole methods or wavelet expansions, is an active area of research today. The black boxes involved in implementing such methods may have thousands of lines of code in them and be based on ideas that only specialists understand.

Projection into Krylov Subspaces

The iterative methods that occupy the remainder of this book are based on the idea of projecting an m-dimensional problem into a lower-dimensional Krylov subspace. Given a matrix A and a vector b, the associated *Krylov sequence* is the set of vectors b, Ab, A^2b, $A^3b, \ldots$, which can be computed by the black box in the form b, Ab, $A(Ab)$, $A(A(Ab)), \ldots$. The corresponding *Krylov subspaces* are the spaces spanned by successively larger groups of these vectors.

Specifically, the algorithms that we shall discuss can be arranged in the following table:

	$Ax = b$	$Ax = \lambda x$
$A = A^*$	CG	Lanczos
$A \neq A^*$	GMRES CGN BCG et al.	Arnoldi

(This field is full of acronyms! CG, for example, stands for conjugate gradients, and, by the way, requires that A be positive definite as well as hermitian.) In each of these methods, the result of projection into the Krylov subspaces is that the original matrix problem is reduced to a sequence of matrix problems of dimensions $n = 1, 2, 3, \ldots$. When A is hermitian, the reduced matrices are tridiagonal, whereas in the nonhermitian case they have Hessenberg form.

Thus the Arnoldi iteration, for example, approximates eigenvalues of a large matrix by computing eigenvalues of certain Hessenberg matrices of successively larger dimensions.

Number of Steps, Work per Step, and Preconditioning

Gaussian elimination, QR factorization, and most other algorithms of dense linear algebra fit the following pattern: there are $O(m)$ steps, each requiring $O(m^2)$ work, for a total work estimate of $O(m^3)$. (Of course these figures, especially the second, may change on a parallel computer.) For iterative methods, the same figures still apply, but now they represent a typical worst-case behavior. When these methods succeed, they may do so by reducing one or both of these factors.

We shall see that the number of steps required for convergence to a satisfactory precision typically depends on spectral properties of the matrix A, if the word "spectral" is interpreted broadly. For example, the conjugate gradient iteration is guaranteed to solve a hermitian positive definite system $Ax = b$ quickly if the eigenvalues of A are clustered well away from the origin. Similarly, the Lanczos iteration is guaranteed to compute certain eigenvalues of a real hermitian matrix quickly if those eigenvalues are well separated from the rest of the spectrum (and if the initial vector that starts the iteration is suitably generic). The analysis of the convergence rates of these methods is a fascinating study that depends on the mathematical field of approximation theory. Specifically, the convergence of Krylov subspace iterative algorithms is closely related to problems of approximation of functions $f(z)$ by polynomials $p(z)$ on subsets of the real axis or the complex plane.

The work per step in a matrix iteration depends mainly on the structure of the matrix and on what advantage is taken of this structure in the $x \mapsto Ax$ black box.

The ideal iterative method in linear algebra reduces the number of steps from m to $O(1)$ and the work per step from $O(m^2)$ to $O(m)$, reducing the total work from $O(m^3)$ to $O(m)$. Such extraordinary speedups do occur in practical problems, but a more typical improvement is perhaps from $O(m^3)$ to $O(m^2)$. In a practical large-scale engineering computation of the mid-1990s, where iterative algorithms are successful, perhaps a typical result is that they beat direct algorithms by a factor on the order of 10. As machines get faster and m gets larger in the future, this factor will increase and iterative algorithms will become more important, illustrating the fundamental law of computer science: the faster the computer, the greater the importance of speed of algorithms.

Exact vs. Approximate Solutions

Matrix iterative methods are approximate in the sense that in principle they do not deliver exact answers, even in the absence of rounding errors, at least

when carried to the number of iterative steps that is of practical interest. This property tends to make newcomers to these ideas uneasy; they may feel that iteration is an "engineering solution" of little elegance and doubtful reliability. This uneasiness tends to diminish as one gets to know these methods better. After all, even direct methods are inexact when carried out on a computer: one hopes for answers accurate to machine precision, no better. Since iterative methods too may be used to achieve the full accuracy of machine precision, the fact that they are in principle approximate need have little significance. As for elegance, the ideas that arise here are some of the most beautiful in numerical linear algebra.

These points are illustrated in Figure 32.1.

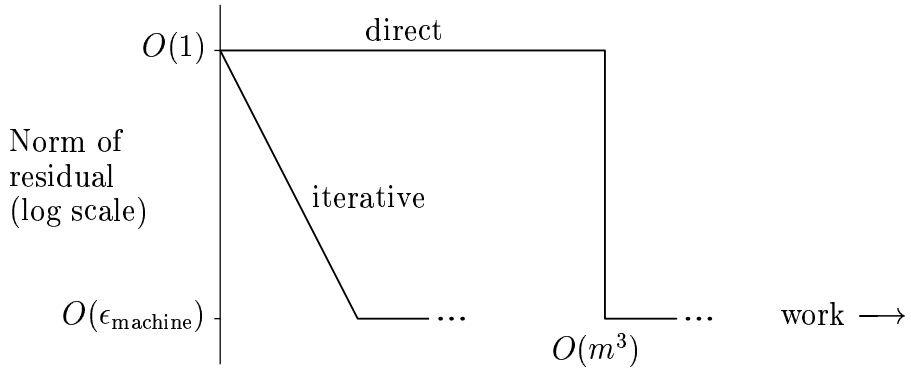


Figure 32.1. *Schematic illustration of convergence of direct and iterative methods. Under favorable circumstances, the iterative method converges geometrically until the residual is on the order of* $\epsilon_{\text{machine}}$. *The direct method makes no progress at all until* $O(m^3)$ *operations are completed, at which point the residual is again on the order of* $\epsilon_{\text{machine}}$.

Direct Methods That Beat $O(m^3)$

Finally, we must mention that there exist direct algorithms—finite, in principle exact—that solve $Ax = b$ and related problems for dense matrices in less than $O(m^3)$ operations. The first algorithm of this kind was discovered in 1969 by Volker Strassen, who reduced Gauss's exponent of 3 to $\log_2(7) \approx 2.81$, and subsequent improvements have reduced the best known exponent to its current value of ≈ 2.376 due to Coppersmith and Winograd. The history of these best known exponents is recorded in Figure 32.2.

So far, these fast algorithms have had negligible impact on practical computation, for two reasons. One is that in general, little is known about their stability properties. More fundamental is the fact that although the exponents in the fast algorithms are striking, the crossover values of m at which

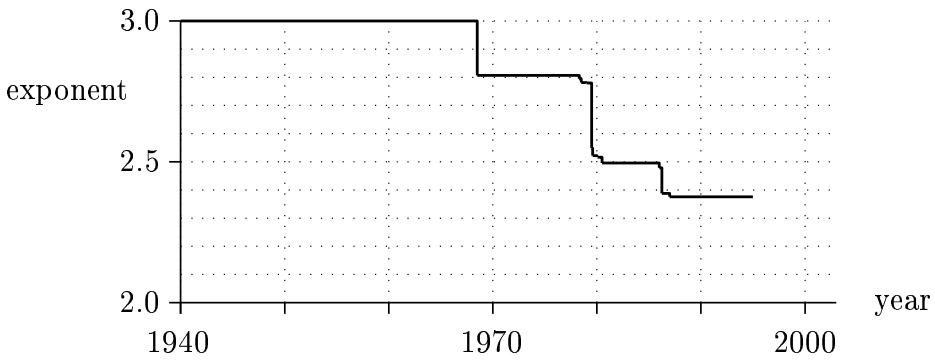


Figure 32.2. *Best known exponents for direct solution of $Ax = b$ (or equivalently, for computation of A^{-1}, AB, or $\det A$) for $m \times m$ matrices, as a function of time. Until* 1968*, the best known algorithms were of complexity $O(m^3)$. The currently best known algorithm solves $Ax = b$ in $O(m^{2.376})$ flops, but the constants are so large that this algorithm is impractical.*

they begin to beat standard methods are exceedingly high. Strassen's $m^{2.81}$ algorithm may be made to beat Gaussian elimination for values of m as low as 100, but because 2.81 is so close to 3, the victory never becomes dramatic for practical values of m. The existing methods with exponents much lower than this involve such large constant factors that they are slower than Gaussian elimination for the values of m attainable on current computers.

But what will happen in the future? The truth is that nobody knows. It is possible that tomorrow somebody will discover a "fast matrix inverse" that solves $Ax = b$ in $m^2 \log m$ floating point operations; or you, the reader, may do so this evening. Such a development would trigger the greatest upheaval in the history of numerical computation.

Exercises

32.1. An elliptic partial differential equation in three dimensions is discretized by a boundary element method. The result is a large dense linear system of equations in which each equation corresponds to a triangular surface element on a large sphere. To improve the accuracy, one must make the triangles smaller and thereby increase the number of equations, but the error shrinks only linearly in proportion to h, the diameter of the largest triangle.

A value of h is chosen, the system is solved by Gaussian elimination, and a solution accurate to two digits is obtained in one minute of computer time. It is decided that three digits of accuracy are needed. Assuming storage is

not a constraint, approximately how much time will be required for the new computation on the same computer?

32.2. Consider the block matrix product

$$\begin{bmatrix} W & X \\ Y & Z \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix},$$

where, for simplicity, all the matrices $A, B, \ldots, Y, Z$ are assumed to be square and of the same dimension.

(a) Given $A, B, \ldots, G, H$, how many (i) matrix additions and (ii) matrix multiplications does it take to compute W, X, Y, Z by the obvious algorithm?

(b) Strassen showed that W, X, Y, Z can also be computed by the formulas

$$\begin{aligned} P_1 &= (A + D)(E + H), & P_5 &= (A + B)H, \\ P_2 &= (C + D)E, & P_6 &= (C - A)(E + F), \\ P_3 &= A(F - H), & P_7 &= (B - D)(G + H), \\ P_4 &= D(G - E), \end{aligned}$$

$$\begin{aligned} W &= P_1 + P_4 - P_5 + P_7, & Y &= P_2 + P_4, \\ X &= P_3 + P_5, & Z &= P_1 + P_3 - P_2 + P_6. \end{aligned}$$

How many (i) matrix additions or subtractions and (ii) matrix multiplications are involved now?

(c) Show that by applying Strassen's formulas recursively, one can obtain an algorithm for multiplying matrices of dimension $m = 2^k$ with an operation count $O(m^{\log_2(7)})$ as $m \to \infty$.

(d) Write a recursive program that implements this idea, and give numerical evidence that your program works.

Lecture 33. The Arnoldi Iteration

Despite the many names and acronyms that have proliferated in the field of Krylov subspace matrix iterations, these algorithms are built upon a common foundation of a few fundamental ideas. One can take various approaches to describing this foundation. Ours will be to consider the Arnoldi process, a Gram–Schmidt-style iteration for transforming a matrix to Hessenberg form.

The Arnoldi/Gram–Schmidt Analogy

Suppose, to pass the time while marooned on a desert island, you challenged yourself to devise an algorithm to reduce a nonhermitian matrix to Hessenberg form by orthogonal similarity transformations, proceeding column by column from a prescribed first column q_1. To your surprise, you would probably find you could solve this problem in an hour and still have time to gather coconuts for dinner. The method you would come up with goes by the name of the Arnoldi iteration. If A is hermitian, the Hessenberg matrix becomes tridiagonal, an n-term recurrence relation becomes a three-term recurrence relation, and the name changes to the Lanczos iteration, to be discussed in Lecture 36.

Here is an analogy. For computing the QR factorization $A = QR$ of a matrix A, we have discussed two methods in this book: Householder reflections, which triangularize A by a succession of orthogonal operations, and Gram–Schmidt orthogonalization, which orthogonalizes A by a succession of triangular operations. Though Householder reflections lead to a more nearly

orthogonal matrix Q in the presence of rounding errors, the Gram–Schmidt process has the advantage that it can be stopped part-way, leaving one with a reduced QR factorization of the first n columns of A. The problem of computing a Hessenberg reduction $A = QHQ^*$ of a matrix A is exactly analogous. There are two standard methods: Householder reflections (applied now on two sides of A rather than one) and the Arnoldi iteration. Thus Arnoldi is the analogue of Gram–Schmidt for similarity transformations to Hessenberg form rather than QR factorization. Like Gram–Schmidt, it has the advantage that it can be stopped part-way, leaving one with a partial reduction to Hessenberg form that is exploited in various manners to form iterative algorithms for eigenvalues or systems of equations.

Thus, this lecture is to Lecture 26 as Lecture 8 is to Lecture 10.

We can summarize the four algorithms just mentioned in a table:

	$A = QR$	$A = QHQ^*$
orthogonal structuring	Householder	Householder
structured orthogonalization	Gram–Schmidt	Arnoldi

For the remainder of this book, m and $n < m$ are positive integers, A is a real or complex $m \times m$ matrix, and $\|\cdot\| = \|\cdot\|_2$. In addition, one further character will now appear in the drama, an m-vector that we shall denote by b. The Arnoldi process needs this vector in order to get started. For applications to eigenvalue problems, we typically assume that b is random. For applications to systems of equations, as considered in later lectures, it will be the right-hand side, or more generally, the initial residual (see Exercise 35.5).

Mechanics of the Arnoldi Iteration

A complete reduction of A to Hessenberg form by an orthogonal similarity transformation might be written $A = QHQ^*$, or $AQ = QH$. However, in dealing with iterative methods we take the view that m is huge or infinite, so that computing the full reduction is out of the question. Instead we consider the first n columns of $AQ = QH$. Let Q_n be the $m \times n$ matrix whose columns are the first n columns of Q:

$$Q_n = \left[\begin{array}{c|c|c|c} & & & \\ q_1 & q_2 & \cdots & q_n \\ & & & \end{array}\right]. \tag{33.1}$$

Here and in the lectures ahead, it would be consistent with our usage elsewhere in the book to put hats on the symbols Q_n, since these matrices are rectangular, but to keep the formulas uncluttered we do not do this.

Let $\tilde{H}_n$ be the $(n+1)\times n$ upper-left section of H, which is also a Hessenberg matrix:

$$\tilde{H}_n = \begin{bmatrix} h_{11} & & \cdots & h_{1n} \\ h_{21} & h_{22} & & \\ & \ddots & \ddots & \vdots \\ & & h_{n,n-1} & h_{nn} \\ & & & h_{n+1,n} \end{bmatrix}. \tag{33.2}$$

Then we have

$$AQ_n = Q_{n+1}\tilde{H}_n, \tag{33.3}$$

that is,

$$\left[\begin{array}{c} \\ A \\ \\ \end{array}\right] \left[\begin{array}{c|c|c} & & \\ q_1 & \cdots & q_n \\ & & \end{array}\right] = \left[\begin{array}{c|c|c} & & \\ q_1 & \cdots & q_{n+1} \\ & & \end{array}\right] \begin{bmatrix} h_{11} & \cdots & h_{1n} \\ h_{21} & & \vdots \\ & \ddots & \\ & & h_{n+1,n} \end{bmatrix}.$$

The nth column of this equation can be written as follows:

$$Aq_n = h_{1n}q_1 + \cdots + h_{nn}q_n + h_{n+1,n}q_{n+1}. \tag{33.4}$$

In words, q_{n+1} satisfies an $(n+1)$-term recurrence relation involving itself and the previous Krylov vectors.

The Arnoldi iteration is simply the modified Gram–Schmidt iteration that implements (33.4). The following algorithm should be compared with Algorithm 8.1.

Algorithm 33.1. Arnoldi Iteration

$b =$ arbitrary, $q_1 = b/\|b\|$
for $n = 1, 2, 3, \ldots$
    $v = Aq_n$
    for $j = 1$ **to** n
        $h_{jn} = q_j^* v$
        $v = v - h_{jn}q_j$
    $h_{n+1,n} = \|v\|$ [see Exercise 33.2 concerning $h_{n+1,n} = 0$]
    $q_{n+1} = v/h_{n+1,n}$

The reader can see at a glance how simple the Arnoldi process is. In a high-level language such as MATLAB, it can be implemented in less than a dozen lines. The matrix A appears only in the product Aq_n, which can be computed by a black box procedure as described in the last lecture.

QR Factorization of a Krylov Matrix

The power of the Arnoldi process lies in the various interpretations that can be made of it, and in the algorithms these suggest. For a first interpretation, consider the recurrence (33.4). It is evident from this formula that the vectors $\{q_j\}$ form bases of the successive *Krylov subspaces* generated by A and b, defined as follows:

$$\mathcal{K}_n \;=\; \langle b, Ab, \ldots, A^{n-1}b\rangle \;=\; \langle q_1, q_2, \ldots, q_n\rangle \;\subseteq\; \mathbb{C}^m\,. \tag{33.5}$$

Moreover, since the vectors q_j are orthonormal, these are orthonormal bases. Thus the Arnoldi process can be described as the systematic construction of orthonormal bases for successive Krylov subspaces.

To express this observation in matrix form, let us define K_n to be the $m\times n$ *Krylov matrix*

$$K_n = \left[\begin{array}{c|c|c|c} & & & \\ b & Ab & \cdots & A^{n-1}b \\ & & & \end{array}\right]. \tag{33.6}$$

Then K_n must have a reduced QR factorization

$$K_n = Q_n R_n, \tag{33.7}$$

where Q_n is the same matrix as above. In the Arnoldi process, neither K_n nor R_n is formed explicitly. Doing so would make for an unstable algorithm, since these are exceedingly ill-conditioned matrices in general, as the columns of K_n all tend to approximate the same dominant eigenvector of A. However, (33.6) and (33.7) give an intuitive explanation of why the Arnoldi process leads to effective methods for determining certain eigenvalues. Clearly K_n might be expected to contain good information about the eigenvalues of A with largest modulus, and the QR factorization might be expected to reveal this information by peeling off one approximate eigenvector after another, starting with the dominant one.

The explanation just given may remind the reader of a similar discussion that appeared earlier in this book. The relationship between (33.6)–(33.7) and the Arnoldi algorithm is analogous to that between simultaneous iteration and the QR algorithm for computing eigenvalues of matrices. One is easy to understand but unstable, the other is subtler but stabler. The difference is

that, whereas the Arnoldi iteration is based upon the QR factorization (33.7) of the matrix whose columns are $b, Ab, \ldots, A^{n-1}b$, simultaneous iteration and the QR algorithm are based upon the QR factorization (28.16) of the matrix whose columns are $A^n e_1, \ldots, A^n e_m$. We can summarize this parallel in another table:

	quasi-direct	iterative
straightforward but unstable	simultaneous iteration	(33.6)–(33.7)
subtle but stable	QR algorithm	Arnoldi

Projection onto Krylov Subspaces

Another way to view the Arnoldi process is as a computation of projections onto successive Krylov subspaces. To see this, note that the product $Q_n^* Q_{n+1}$ is the $n \times (n+1)$ identity, i.e., the $n \times (n+1)$ matrix with 1 on the main diagonal and 0 elsewhere. Therefore $Q_n^* Q_{n+1} \tilde{H}_n$ is the $n \times n$ Hessenberg matrix obtained by removing the last row of $\tilde{H}_n$:

$$H_n = \begin{bmatrix} h_{11} & & \cdots & h_{1n} \\ h_{21} & h_{22} & & \\ & \ddots & \ddots & \vdots \\ & & h_{n,n-1} & h_{nn} \end{bmatrix}. \tag{33.8}$$

From (33.3) we accordingly have

$$H_n = Q_n^* \, A \, Q_n. \tag{33.9}$$

This matrix can be interpreted as the representation in the basis $\{q_1, \ldots, q_n\}$ of the orthogonal projection of A onto $\mathcal{K}_n$. Is it clear what this interpretation means? Here is a precise statement. Consider the linear operator $\mathcal{K}_n \to \mathcal{K}_n$ defined as follows: given $v \in \mathcal{K}_n$, apply A to it, then orthogonally project Av back into the space $\mathcal{K}_n$. Since the orthogonal projector of $\mathbb{C}^m$ onto $\mathcal{K}_n$ is $Q_n Q_n^*$, this operator can be written $Q_n Q_n^* A$ with respect to the standard basis of $\mathbb{C}^m$. With respect to the basis of columns of Q_n, it can therefore be written $Q_n^* A \, Q_n$.

The kind of projection just described comes up throughout applied and numerical mathematics. In another context it is known as the *Rayleigh–Ritz* procedure; not coincidentally, in the diagonal elements of H_n one recognizes the Rayleigh quotients of A with respect to the vectors q_j. This projection process is also one of the ideas underlying finite element methods for solution

of partial differential equations, as well as their younger relatives known as spectral methods.

Since H_n is a projection of A, one might imagine that its eigenvalues would be related to those of A in a useful fashion. These n numbers,

$$\{\theta_j\} = \{\text{eigenvalues of } H_n\}, \tag{33.10}$$

are called the *Arnoldi eigenvalue estimates* (at step n) or *Ritz values* (with respect to $\mathcal{K}_n$) of A. In the next lecture we shall see that some of these numbers may be extraordinarily accurate approximations to some of the eigenvalues of A, even for $n \ll m$.

We summarize the developments of this lecture in a theorem, to be compared with Theorem 28.3.

Theorem 33.1. *The matrices Q_n generated by the Arnoldi iteration are reduced QR factors of the Krylov matrix* (33.6)*:*

$$K_n = Q_n R_n. \tag{33.11}$$

The Hessenberg matrices H_n are the corresponding projections

$$H_n = Q_n^* A Q_n, \tag{33.12}$$

and the successive iterates are related by the formula

$$AQ_n = Q_{n+1} \tilde{H}_n. \tag{33.13}$$

Exercises

33.1. Let $A \in \mathbb{C}^{m\times m}$ and $b \in \mathbb{C}^m$ be arbitrary. Show that any $x \in K_n$ is equal to $p(A)b$ for some polynomial p of degree $\le n-1$.

33.2. Suppose Algorithm 33.1 is executed for a particular A and b until at some step n, an entry $h_{n+1,n} = 0$ is encountered.

(a) Show how (33.13) can be simplified in this case. What does this imply about the structure of a full $m \times m$ Hessenberg reduction $A = QHQ^*$ of A?

(b) Show that $\mathcal{K}_n$ is an invariant subspace of A, i.e., $A\mathcal{K}_n \subseteq \mathcal{K}_n$.

(c) Show that if the Krylov subspaces of A generated by b are defined by $\mathcal{K}_k = \langle b, Ab, \dots, A^{k-1}b\rangle$ as in (33.5), then $\mathcal{K}_n = \mathcal{K}_{n+1} = \mathcal{K}_{n+2} = \cdots$.

(d) Show that each eigenvalue of H_n is an eigenvalue of A.

(e) Show that if A is nonsingular, then the solution x to the system of equations $Ax = b$ lies in $\mathcal{K}_n$.

The appearance of an entry $h_{n+1,n} = 0$ is called a *breakdown* of the Arnoldi iteration, but it is a breakdown of a benign sort. For applications in computing eigenvalues (Lecture 34) or solving systems of equations (Lecture 35), because of (d) and (e), a breakdown usually means that convergence has occurred and the iteration can be terminated. Alternatively, a new orthonormal vector q_{n+1} could be selected at random and the iteration then continued.

33.3. (a) Suppose Algorithm 33.1 is executed for a particular A and b and runs to completion ($n = m$), with no breakdown of the kind described in the last exercise. Show that this implies that the minimal polynomial of A is of degree m.

(b) Conversely, suppose that the minimal polynomial of A is of degree m. Show that this does not imply that for a particular choice of b, Algorithm 33.1 will necessarily run to completion.

(c) Explain why the result of (a) does not contradict Exercise 25.1(b).

Lecture 34. How Arnoldi Locates Eigenvalues

The Arnoldi iteration is two things: the basis of many of the iterative algorithms of numerical linear algebra and, more specifically, a technique for finding eigenvalues of nonhermitian matrices. Here we consider this second, specific role, and in the process, we describe a connection with polynomial approximation theory that is of broad importance.

Computing Eigenvalues by the Arnoldi Iteration

The use of the Arnoldi iteration for computing eigenvalues proceeds as follows. The iteration is carried out as described in the last lecture (Algorithm 33.1). At each step n, or at occasional steps, the eigenvalues of the Hessenberg matrix H_n are computed by standard methods such as the QR algorithm. (In practice this means a call to software such as provided by EISPACK, LAPACK, or MATLAB.) These are the "Arnoldi estimates" or "Ritz values" (33.10). Some of these numbers are typically observed to converge rapidly, often geometrically (i.e., linearly; see Exercise 25.2), and when they do, one may assume with reasonable confidence that the converged values are eigenvalues of A.

Since $n \ll m$ for a feasible computation, one cannot of course expect to compute all of the eigenvalues of A by this process. Which eigenvalues, then, does the Arnoldi iteration find? Typically, it finds *extreme* eigenvalues, that is, eigenvalues near the edge of the spectrum of A. Fortunately, these are precisely the eigenvalues of main interest in most applications.

For example, in problems of hydrodynamic stability, the aim is to determine whether small perturbations to a smooth fluid flow may grow unstably, in which case the flow is likely to break down into another form that is more complicated, perhaps turbulent. This determination of stability is conventionally achieved by linearizing the problem to obtain a Jacobian operator governing evolution of small perturbations, then calculating the rightmost eigenvalue of this operator in the complex plane, that is, the eigenvalue with maximal real part. If this eigenvalue is in the right half-plane, the flow is unstable, whereas if it is in the left half-plane, it is stable.

Many variations on the Arnoldi method have been devised, such as acceleration devices for problems in which some parts of the complex plane are of greater importance than others. Here we confine our attention to the unadorned Arnoldi method, attempting to give an idea of why it converges, why it tends to find extreme eigenvalues of A, and how quickly it finds them.

A Note of Caution: Nonnormality

First, however, we call the reader's attention to a cautionary fact. The physical significance of the eigenvalues of nonhermitian matrices is sometimes not as great as supposed. If a matrix is far from normal—that is, if its eigenvectors are far from orthogonal, which implies that its eigenvalues are ill-conditioned—then the eigenvalues may have little to do with how a physical system governed by the matrix actually behaves (see Exercises 24.3 and 26.2). In the problem of fluid flow through a circular pipe, for example, the Jacobian of the linearized equations has all its eigenvalues in the left half-plane, suggesting that the flow ought to be stable. In actuality, however, high-speed flows through pipes are invariably turbulent. The resolution of this paradox is rooted in the fact that although the linearized problem has all decaying eigenmodes, it may nevertheless amplify certain nonmodal flow perturbations by many orders of magnitude. Such phenomena cannot occur with normal matrices or operators—e.g., symmetric, hermitian, or self-adjoint.

If the answer is highly sensitive to perturbations, you have probably asked the wrong question. We urge anyone faced with nonhermitian eigenvalue computations involving highly sensitive eigenvalues to bear this principle in mind. If you are a numerical analyst, and the problem was given to you by a colleague in science or engineering, do not accept without scrutiny your colleague's assurances that it is truly the eigenvalues that matter physically, even though their condition numbers with respect to perturbations of the matrix entries are 10^4. Perhaps situations exist where highly sensitive eigenvalues of nonnormal operators are of genuine physical significance, but they are outnumbered by situations where eigenvalues are mistakenly investigated when a deeper analysis is properly called for.

Arnoldi and Polynomial Approximation

Let x be a vector in the Krylov subspace $\mathcal{K}_n$ (33.5). Such an x can be written as a linear combination of powers of A times b:

$$x = c_0 b + c_1 Ab + c_2 A^2 b + \cdots + c_{n-1} A^{n-1} b. \tag{34.1}$$

This expression has a compact description: it is a *polynomial in* A times b. That is, if q is the polynomial $q(z) = c_0 + c_1 z + \cdots + c_{n-1} z^{n-1}$, then we have

$$x = q(A)b, \tag{34.2}$$

as pointed out already in Exercise 33.1. Because every vector of the form (34.1) can be expressed in the form (34.2), Krylov subspace iterations can always be analyzed in terms of matrix polynomials.

One analysis of the Arnoldi iteration takes the following form. Define

$$P^n = \{\text{monic polynomials of degree } n\}.$$

(The word "monic" means that the coefficient of the term of degree n is 1.) Now consider the following approximation problem.

Arnoldi/Lanczos Approximation Problem. Find $p^n \in P^n$ such that

$$\| p^n(A)b \| = \text{minimum}. \tag{34.3}$$

As always in this part of the book, $\| \cdot \| = \| \cdot \|_2$. The reason we have used superscripts on P^n is that in the next lecture, we shall consider the space of degree-n polynomials normalized by $c_0 = 1$ rather than $c_n = 1$, and for this space we shall use a subscript.

The Arnoldi iteration has the remarkable property that it solves (34.3) exactly.

Theorem 34.1. *As long as the Arnoldi iteration does not break down (i.e., K_n is of full rank n),* (34.3) *has a unique solution p^n, namely, the characteristic polynomial of H_n.*

Proof. First we note that if $p \in P^n$, then the vector $p(A)b$ can be written $p(A)b = A^n b - Q_n y$ for some $y \in \mathbb{C}^n$, where Q_n is defined by (33.1). In other words, (34.3) is equivalent to a linear least squares problem: find the point in $\mathcal{K}_n$ closest to $A^n b$, or in matrix terms, find y such that $\|A^n b - Q_n y\|$ is minimal. The solution is characterized by the orthogonality condition $p^n(A)b \perp \mathcal{K}_n$, illustrated in Figure 34.1, or equivalently, $Q_n^* \, p^n(A)b = 0$. Now consider the factorization $A = QHQ^*$ mentioned at the beginning of the last lecture. At

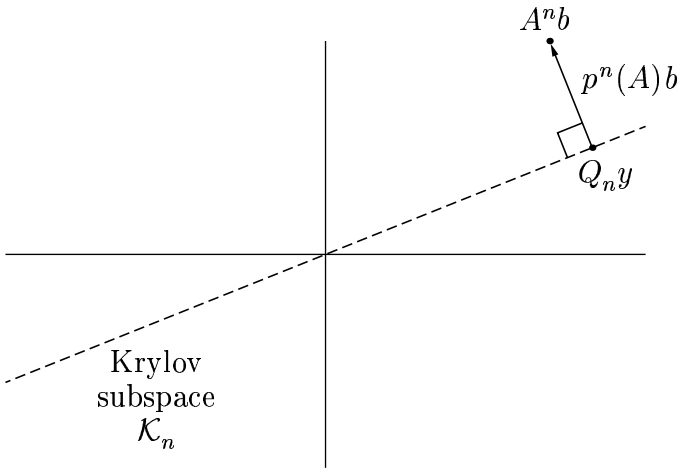


Figure 34.1. *The least squares polynomial approximation problem underlying the Arnoldi iteration.*

step n of the Arnoldi process, we have computed the first n columns of Q and H; thus we know that a factorization of this kind exists with

$$Q = \begin{bmatrix} Q_n & U \end{bmatrix}, \qquad H = \begin{bmatrix} H_n & X_1 \\ X_2 & X_3 \end{bmatrix}$$

for some $m \times (m-n)$ matrix U with orthonormal columns that are also orthogonal to the columns of Q_n and some matrices X_1, X_2, and X_3 of dimensions $n \times (m-n)$, $(m-n) \times n$, and $(m-n) \times (m-n)$, respectively, with all but the upper-right entry of X_2 equal to 0. The orthogonality condition becomes $Q_n^* Q\, p^n(H)\, Q^* b = 0$, which amounts to the condition that the first n entries of the first column of $p^n(H)$ are zero (since only the first entry of $Q^* b$ is nonzero, as noted below on p. 268). Because of the structure of H, these are also the first n entries of the first column of $p^n(H_n)$. By the Cayley–Hamilton theorem, these are zero if p^n is the characteristic polynomial of H_n. Conversely, suppose there were another polynomial p^n with $p^n(A)\,b \perp \mathcal{K}_n$. Taking the difference would give a nonzero polynomial q of degree $n-1$ with $q(A)\,b = 0$, violating the assumption that K_n is of full rank. □

Theorem 34.1 gives us a new interpretation of the Ritz values generated by the Arnoldi iteration. They are the roots of the optimal polynomial (34.3).

Invariance Properties

Theorem 34.1 provides an easy way to remember some of the basic properties of the Arnoldi iteration. For example, since the family P^n of monic polynomials is invariant with respect to translations $z \mapsto z + \alpha$, the Arnoldi iteration is also translation-invariant. Unlike iterative algorithms for solving $Ax = b$, to be discussed later, such as GMRES, it does not "know where the origin is."

Here is a summary of this and other invariance properties. Each part of this theorem is a corollary of Theorem 34.1; we do not give proofs.

Theorem 34.2. *Let the Arnoldi iteration be applied to a matrix $A \in \mathbb{C}^{m\times m}$ as described above.*

Translation-invariance. *If A is changed to $A + \sigma I$ for some $\sigma \in \mathbb{C}$, and b is left unchanged, the Ritz values $\{\theta_j\}$ at each step change to $\{\theta_j + \sigma\}$.*

Scale-invariance. *If A is changed to σA for some $\sigma \in \mathbb{C}$, and b is left unchanged, the Ritz values $\{\theta_j\}$ change to $\{\sigma\theta_j\}$.*

Invariance under unitary similarity transformations. *If A is changed to UAU^* for some unitary matrix U, and b is changed to Ub, the Ritz values $\{\theta_j\}$ do not change.*

In all three cases the Ritz vectors*, namely the vectors $Q_n y_j$ corresponding to the eigenvectors y_j of H_n, do not change under the indicated transformation.*

By Theorem 24.9, every matrix A is unitarily similar to an upper-triangular matrix. Thus the property of invariance under unitary similarity transformations implies that for understanding convergence properties of the Arnoldi iteration, it is enough in principle to consider upper-triangular matrices. It is not, however, enough to consider diagonal matrices. In the nonhermitian (nonnormal) case, there is more to a matrix than its eigenvalues.

How Arnoldi Locates Eigenvalues

We can now begin to address the problem raised in the title of this lecture. Theorem 34.1 asserts that at bottom, the "goal" of the Arnoldi iteration is to solve a polynomial approximation problem, or equivalently, a least squares problem involving a Krylov subspace. If the Arnoldi iteration tends to find eigenvalues, it must be as a by-product of achieving this goal.

What does polynomial approximation have to do with the eigenvalues of A? A little thought shows that there is a connection between these two along the following lines. If one's aim is to find a polynomial p^n with the property that $p^n(A)$ is small, an effective means to that end may be to pick p^n to have zeros close to the eigenvalues of A.

Consider an extreme case. Suppose that A is diagonalizable and has only $n \ll m$ distinct eigenvalues, hence a minimal polynomial of degree n. Then from Theorem 34.1 it is clear that after n steps, all of these eigenvalues will be found exactly, at least if the vector b contains components in directions associated with every eigenvalue. Thus after n steps, the Arnoldi iteration has computed the minimal polynomial of A exactly.

In practical applications, much the same phenomenon takes place. Now, however, the agreement of Ritz values with eigenvalues is approximate instead of exact, and instead of a minimal polynomial, the result is a "pseudo-minimal polynomial," i.e., a polynomial p^n such that $\|p^n(A)\|$ is small.

Arnoldi Lemniscates

This convergence process can be illustrated graphically by plotting lemniscates in the complex plane. A *lemniscate* is a curve or collection of curves

$$\{z \in \mathbb{C} : \ |p(z)| = C\}, \tag{34.4}$$

where p is a polynomial and C is a real constant. If p is the Arnoldi polynomial p^n for an Arnoldi iteration with matrix A at step n, and C takes the value

$$C \ = \ \frac{\|p^n(A)\,b\|}{\|b\|}, \tag{34.5}$$

then the curves defined by (34.4) may be called *Arnoldi lemniscates*. As the iteration number n increases, components of these lemniscates typically appear which surround the extreme eigenvalues of A and then shrink rapidly to a point, namely the eigenvalue itself.

To illustrate these ideas, let A be a square matrix of dimension $m = 100$ whose entries are independent random numbers from the real normal distribution of mean 0 and standard deviation $m^{-1/2}$ (Exercise 12.3). Since A is real, its eigenvalues consist of real numbers and complex conjugate pairs. Our choice of the standard deviation is such that the eigenvalues are approximately uniformly distributed in the unit disk $|z| \le 1$. But now we create an outlier eigenvalue by changing the corner entry a_{11} to 1.5. See Figure 34.2.

We apply the Arnoldi iteration to this matrix, beginning with a random vector q_1. At step n, the matrix H_n has been constructed and its characteristic polynomial reflects the iteration's current knowledge about the spectrum of A. Figure 34.3 plots Arnoldi lemniscates at steps $n = 5, 6, 7, 8$. At $n = 1$ (not shown), the lemniscate is an exact circle, little affected by the outlying eigenvalue. By $n = 5$, the circle has begun to bulge in the direction of λ. At $n = 6$, the bulge pinches off and a new component of the lemniscate appears. This component then proceeds to shrink with each subsequent iteration.

Geometric Convergence

Under certain circumstances, the convergence of some of the Arnoldi eigenvalue estimates to eigenvalues of A is geometric. These matters are incompletely understood at present, and we shall not cite theoretical results here. Instead, we shall just take a look at the convergence in the numerical example above, and give a partial explanation.

From a figure like Figure 34.3, one cannot assess the rate of convergence beyond one or two digits of accuracy. Figure 34.4 fills this gap by plotting $|\lambda^{(n)} - \lambda|$ as a function of n for the same example as before, where $\lambda^{(n)}$ is the Arnoldi eigenvalue estimate closest to the outlier eigenvalue λ at step n. The first thing we notice is that the convergence is clearly geometric. After fifty

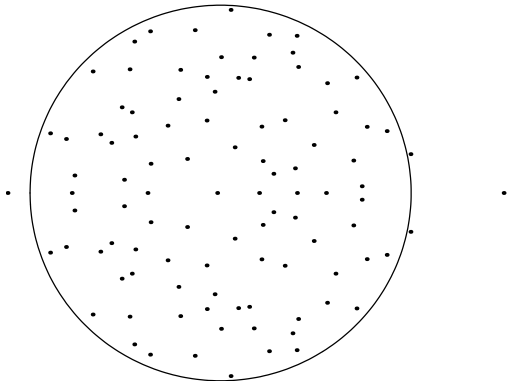


Figure 34.2. *Eigenvalues of a* 100×100 *matrix* A*, random except in the* $1,1$ *position. The circle is the unit circle in* $\mathbb{C}$*. The eigenvalues are approximately uniformly distributed in the unit disk except for the outlier* $\lambda \approx 1.4852$*.*

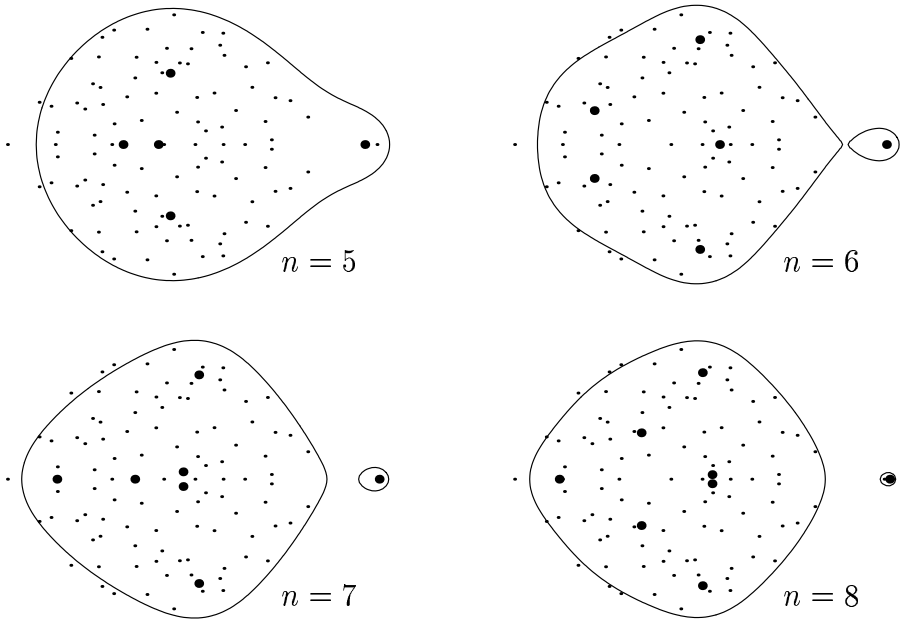


Figure 34.3. *Arnoldi lemniscates* (34.4)–(34.5) *at steps* $n = 5, 6, 7, 8$ *for the same matrix* A*. The small dots are the eigenvalues of* A*, and the large dots are the eigenvalues of* H_n*, i.e., the Ritz values. One component of the Arnoldi lemniscate first "swallows" the outlier eigenvalue, and in subsequent iterations it then shrinks to a point at a geometric rate.*

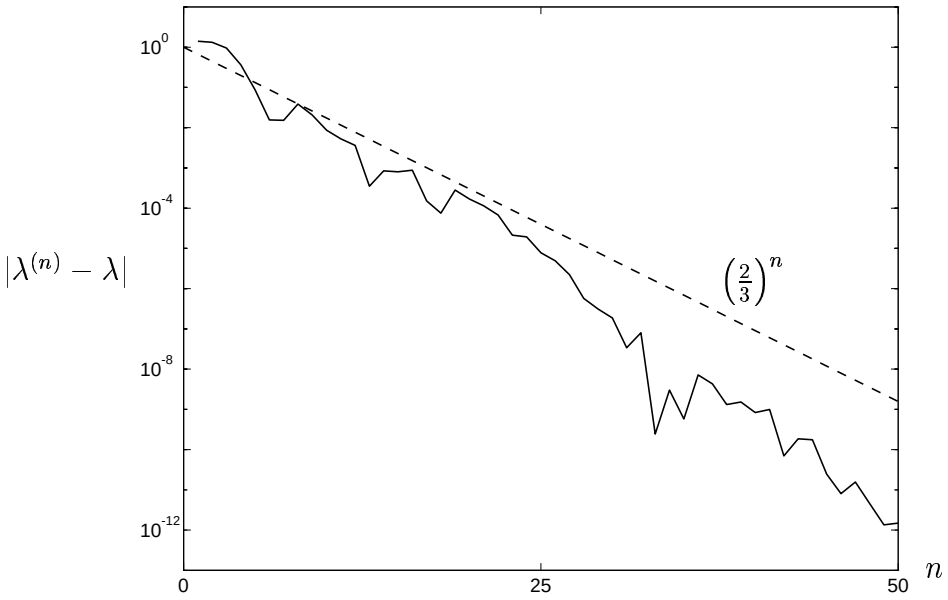


Figure 34.4. *Convergence of the rightmost Arnoldi eigenvalue estimate.*

iterations, we have achieved twelve digits of precision, and this figure would not have been much different if A had had dimension 1000 instead of 100.

We can be more quantitative. For the first few dozen iterations, at least, the convergence in Figure 34.3 approximates the rate

$$|\lambda^{(n)} - \lambda| \approx \left(\frac{2}{3}\right)^n. \tag{34.6}$$

An explanation of this behavior is as follows. To minimize (34.3), p^n must take a roughly minimal value at each of the eigenvalues of A. Consider for example the candidate polynomial $p(z) = z^{n-1}(z-\tilde{\lambda})$, where $\tilde{\lambda}$ is some number close to λ. At each of the eigenvalues of A in the unit disk, $|p(z)|$ is of order 1 or smaller. At $z = \lambda$, however, it has magnitude

$$|p(\lambda)| \approx \left(\frac{3}{2}\right)^n |\tilde{\lambda} - \lambda|$$

(this would be an equality if λ were exactly equal to 3/2). When n is large, $(3/2)^n$ is huge. For this number also to be of order 1, $|\tilde{\lambda} - \lambda|$ must be small enough to balance it, that is, of order $(2/3)^n$, as in (34.6).

Another feature is apparent in Figure 34.4. After the initial few dozen steps, the convergence begins to accelerate, a phenomenon common in Krylov subspace iterations. What is happening here is that the iteration is beginning to resolve some of the other outer eigenvalues of A, near the unit circle. If the dimension of A had been $m = 300$, then the cloud of eigenvalues would have filled the unit disk sufficiently densely that no such acceleration would have been visible in the fifty iterative steps shown in Figure 34.4.

Exercises

34.1. Given $A \in \mathbb{C}^{m\times m}$, $b \in \mathbb{C}^m$, and $p \in P^n$, suppose we want to compute $p(A)b$. A natural place to start is with Horner's rule, which can be written

$$p(z) \;=\; c_0 + z(c_1 + z(c_2 + \cdots + z(c_{n-1} + z)\cdots)). \tag{34.7}$$

(a) Write a **for** loop based on (34.7) (on paper, not on a computer) that computes $p(A)$ and then applies the result to b. Determine the number of flops required by this algorithm, to leading order.

(b) Write another **for** loop for computing $p(A)b$, a far better one, in which b is introduced into the process at the beginning. Again determine the number of flops required to leading order.

(c) In introductory numerical analysis texts, Horner's rule is recommended for evaluation of polynomials because it is faster than the obvious method of computing powers z^k, multiplying by coefficients c_k, and adding. Show that for computing $p(A)$ or $p(A)b$, by contrast, Horner's rule is not significantly faster than the obvious method.

34.2. We have seen that the Arnoldi polynomial p^n minimizes $\|p^n(A)\,b\|$. Another polynomial that might give cleaner information about eigenvalues would be the *ideal Arnoldi polynomial*, also known as the *Chebyshev polynomial of* A, defined as the unique $p^* \in P^n$ that minimizes $\|p^n(A)\|$. (This polynomial is not used in practice, because there is no fast way to compute it.)

(a) Prove that p^* exists.

(b) Prove that provided $p^*(A) \neq 0$, p^* is unique. (Hint: Suppose p_1 and p_2 are distinct ideal Arnoldi polynomials for given A and n, set $p = (p_1 + p_2)/2$, and consider the singular vectors of $p(A)$ corresponding to the maximal singular value. This is a hard problem.)

34.3. Let A be the $N \times N$ bidiagonal matrix with $a_{k,k+1} = a_{k,k} = k^{-1/2}$, $N = 64$. (In the limit $N \to \infty$, A becomes a non-self-adjoint compact operator.)

(a) Produce a plot showing the spectrum $\Lambda(A)$ and the boundaries of the ϵ-pseudospectra $\Lambda_\epsilon(A)$ (Exercises 26.1 and 26.2) for $\epsilon = 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}$.

(b) Starting from a random initial vector, run the Arnoldi iteration and compute Ritz values at steps $n = 1, 2, \ldots, 30$. Produce plots to indicate the rates of convergence to eigenvalues of A, and comment on your results.

(c) The Arnoldi iteration can also be used to approximate pseudospectra of A by those of H_n or $\tilde{H}_n$. (In the latter case, the boundary of $\Lambda_\epsilon(\tilde{H}_n)$ is defined by the condition $\sigma_{\min}(zI - \tilde{H}_n) = \epsilon$, or equivalently $\|(zI - \tilde{H}_n)^+\| = \epsilon^{-1}$, where I is a rectangular version of the identity.) Experiment with this idea by plotting the ϵ-pseudospectra of $\tilde{H}_n$ for $n = 5, 10, 15, 20$. How closely do they match the corresponding pseudospectra of A?

Lecture 35. GMRES

In the last lecture we showed how the Arnoldi process can be used to find eigenvalues. Here we show that it can also be used to solve systems of equations $Ax = b$. The standard algorithm of this kind is known as GMRES, which stands for "generalized minimal residuals."

Residual Minimization in $\mathcal{K}_n$

As in the last two lectures, let $A \in \mathbb{C}^{m\times m}$ be a square matrix and $b \in \mathbb{C}^m$ a vector, and let $\mathcal{K}_n$ denote the Krylov subspace $\langle b, Ab, \ldots, A^{n-1}b\rangle$ of (33.5). Now, however, we assume that A is nonsingular, for our goal is to solve a system of equations $Ax = b$. It will be convenient to have a notation for the exact solution of this problem: $x_* = A^{-1}b$.

The idea of GMRES is a one-liner. At step n, we shall approximate x_* by the vector $x_n \in \mathcal{K}_n$ that minimizes the norm of the residual $r_n = b - Ax_n$. In other words, we shall determine x_n by solving a least squares problem, illustrated in Figure 35.1.

The obvious way to solve this least squares problem would be as follows.

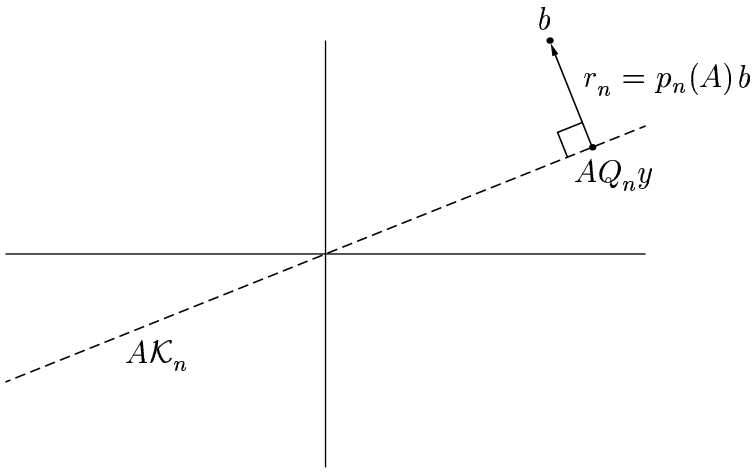


Figure 35.1. *The least squares polynomial approximation problem underlying GMRES: minimize the residual norm* $\|r_n\|$. *Compare Figure* 34.1.

Let K_n be the $m \times n$ Krylov matrix (33.6), so that we have

$$AK_n = \left[\begin{array}{c|c|c|c} Ab & A^2b & \cdots & A^nb \end{array}\right]. \tag{35.1}$$

The column space of this matrix is $A\mathcal{K}_n$. Thus our problem is to find a vector $c \in \mathbb{C}^n$ such that

$$\|AK_nc - b\| = \text{minimum}, \tag{35.2}$$

where $\|\cdot\| = \|\cdot\|_2$, as always in this part of the book. This could be done by means of a QR factorization of AK_n, in analogy to (33.7). Once c is found, we would set $x_n = K_nc$.

The procedure just described is numerically unstable, however, and it constructs a factor R that is not needed. Here is what is actually done instead. We use the Arnoldi iteration (Algorithm 33.1) to construct a sequence of Krylov matrices Q_n whose columns $q_1, q_2, \ldots$ span the successive Krylov subspaces $\mathcal{K}_n$. Thus we write $x_n = Q_ny$ instead of $x_n = K_nc$. Instead of (35.2), our least squares problem is to find a vector $y \in \mathbb{C}^n$ such that

$$\|AQ_ny - b\| = \text{minimum}. \tag{35.3}$$

Superficially, the problem (35.3) has dimensions $m \times n$. In actuality, however, because of the special structure of Krylov subspaces, it is essentially of dimensions $(n+1) \times n$. We can reveal this as follows. Applying (33.3) transforms the equation to

$$\|Q_{n+1}\tilde{H}_ny - b\| = \text{minimum}. \tag{35.4}$$

Now both vectors inside the norm are in the column space of Q_{n+1}. Therefore, multiplying on the left by Q_{n+1}^* does not change that norm. Thus a further equivalent problem is

$$\|\tilde{H}_n y - Q_{n+1}^* b\| = \text{minimum}. \tag{35.5}$$

Finally, we note that by construction of the Krylov matrices $\{Q_n\}$, $Q_{n+1}^* b$ is equal to $\|b\|e_1$, where $e_1 = (1, 0, 0, \ldots)^*$ as usual. Thus we reach at last the final form of the GMRES least squares problem:

$$\|\tilde{H}_n y - \|b\|e_1\| = \text{minimum}. \tag{35.6}$$

At step n of GMRES we solve this problem for y, then set $x_n = Q_n y$.

Mechanics of GMRES

This completes our derivation of the GMRES algorithm. A high-level description is as follows.

Algorithm 35.1. GMRES

$q_1 = b/\|b\|$
for $n = 1, 2, 3, \ldots$
    $\langle$ *step n of Arnoldi iteration, Algorithm* 33.1 $\rangle$
    Find y to minimize $\|\tilde{H}_n y - \|b\|e_1\|$ $(= \|r_n\|)$
    $x_n = Q_n y$.

At each step, GMRES minimizes the norm of the residual $r_n = b - Ax_n$ over all vectors $x_n \in \mathcal{K}_n$. The quantity $\|r_n\|$ is computed in the course of finding y; one need not calculate it explicitly from x_n.

The inner, "Find y" step of Algorithm 35.1 is an $(n+1) \times n$ matrix least squares problem with Hessenberg structure. It can be solved via QR factorization in the usual manner described in Lecture 11, at a cost of $O(n^2)$ flops, thanks to the Hessenberg structure. In addition, it is possible to save further work by a more specialized approach. Rather than construct QR factorizations of the successive matrices $\tilde{H}_1, \tilde{H}_2, \ldots$ independently, one can use an updating process to get the QR factorization of $\tilde{H}_n$ from that of $\tilde{H}_{n-1}$. All that is required is a single Givens rotation (Exercise 10.4) and $O(n)$ work. See Exercise 35.4.

GMRES and Polynomial Approximation

In the last lecture, we showed that the calculation of eigenvalues by the Arnoldi iteration is related to the approximation problem (34.3): find $p^n \in P^n$ to minimize $\|p^n(A)b\|$, where P^n denotes the set of monic polynomials of degree n.

The GMRES iteration also solves an approximation problem, the only difference being that the space of polynomials is now

$$P_n = \{\text{polynomials } p \text{ of degree} \le n \text{ with } p(0) = 1\}. \tag{35.7}$$

Expressed in terms of polynomial coefficients, our normalization is now $c_0 = 1$ rather than $c_n = 1$. Notationally, we have changed from a superscript to a subscript as a reminder of the change from normalization at $z = \infty$ to normalization at $z = 0$.

Here is how GMRES can be reduced to polynomial approximation in P_n. The iterate x_n can be written

$$x_n = q_n(A)b, \tag{35.8}$$

where q is a polynomial of degree $n-1$; its coefficients are the entries of the vector c of (35.2). The corresponding residual $r_n = b - Ax_n$ is $r_n = (I - Aq_n(A))b$, where p_n is the polynomial defined by $p_n(z) = 1 - zq(z)$. In other words, we have

$$r_n \;=\; p_n(A)b \tag{35.9}$$

for some polynomial $p_n \in P_n$. The GMRES process chooses the coefficients of p_n to minimize the norm of this residual. Thus we have shown that GMRES solves the following approximation problem successively for $n = 1, 2, 3 \ldots$.

GMRES Approximation Problem. Find $p_n \in P_n$ such that

$$\|p_n(A)b\| = \text{minimum}. \tag{35.10}$$

Like the Arnoldi iteration, GMRES satisfies certain invariance properties. The following theorem is easily proved from (35.10).

Theorem 35.1. *Let the GMRES iteration be applied to a matrix $A \in \mathbb{C}^{m\times m}$ as described above.*

Scale-invariance. *If A is changed to σA for some $\sigma \in \mathbb{C}$, and b is changed to σb, the residuals $\{r_n\}$ change to $\{\sigma r_n\}$.*

Invariance under unitary similarity transformations. *If A is changed to UAU^* for some unitary matrix U, and b is changed to Ub, the residuals $\{r_n\}$ change to $\{U^* r_n\}$.*

GMRES is not invariant under translation, since the normalization $p(0) = 1$ involves the translation-dependent point 0. On the contrary, its behavior depends strongly on the position of the origin—loosely speaking, on the condition number of A.

Convergence of GMRES

How quickly does GMRES converge? How many steps n must be taken before $\|r_n\|/\|b\|$ is reduced to a satisfactory level such as 10^{-3} or 10^{-16}? As a practical matter, this often becomes the question of designing a good preconditioner (Lecture 40). Mathematically, the problem is to investigate what properties of A determine the size of $\|r_n\|$.

We begin with two observations. The first is that GMRES converges monotonically:

$$\|r_{n+1}\| \le \|r_n\|. \tag{35.11}$$

The reason is that $\|r_n\|$ is as small as possible for the subspace $\mathcal{K}_n$; by enlarging $\mathcal{K}_n$ to the space $\mathcal{K}_{n+1}$, we can only decrease the residual norm, or at worst leave it unchanged. (Note that $P_n \subseteq P_{n+1}$, whereas $P^n \not\subseteq P^{n+1}$.) The second is that after at most m steps the process must converge, at least in the absence of rounding errors:

$$\|r_m\| = 0. \tag{35.12}$$

For generic data A and b, this will happen because $\mathcal{K}_m = \mathbb{C}^m$, and in special cases, if x_* happens to lie in $\mathcal{K}_n$ for some $n < m$, convergence will occur earlier. Equation (35.12) is useful as a reminder of the mathematics of GMRES, but it has little practical content, since a GMRES iteration must converge to satisfactory precision in $n \ll m$ steps if it is to be useful.

To obtain more useful information about convergence, we must turn to the polynomial approximation problem (35.10). We know that $\|r_n\| = \|p_n(A)b\| \le \|p_n(A)\|\|b\|$ is minimal. Except for problems where the right-hand side b has special structure related to that of A, the critical factor that determines the size of this quantity is usually $\|p_n(A)\|$. That is, what determines the convergence rate of GMRES is generally the inequality

$$\frac{\|r_n\|}{\|b\|} \le \inf_{p_n \in P_n} \|p_n(A)\|. \tag{35.13}$$

This brings us to the mathematically elegant question: given a matrix A and a number n, how small can $\|p_n(A)\|$ be? This question is the basis of almost all analyses of convergence of Krylov subspace iterations for solving systems of equations.

Polynomials Small on the Spectrum

Given A and n, how small can $\|p_n(A)\|$ be? The standard way of obtaining estimates is to look for polynomials $p(z)$ that are as small as possible on the spectrum $\Lambda(A)$, while still satisfying $p(0) = 1$. If p is a polynomial and S is a set in the complex plane, let us define the scalar $\|p\|_S$ by

$$\|p\|_S = \sup_{z \in S} |p(z)|. \tag{35.14}$$

Suppose A is diagonalizable, satisfying $A = V\Lambda V^{-1}$ for some nonsingular matrix V and diagonal matrix Λ. Then we have

$$\|p(A)\| \le \|V\|\|p(\Lambda)\|\|V^{-1}\| = \kappa(V)\|p\|_{\Lambda(A)}. \tag{35.15}$$

Combining this result with (35.13) gives the following basic theorem on convergence of GMRES.

Theorem 35.2. *At step n of the GMRES iteration, the residual r_n satisfies*

$$\frac{\|r_n\|}{\|b\|} \le \inf_{p_n \in P_n} \|p_n(A)\| \le \kappa(V) \inf_{p_n \in P_n} \|p_n\|_{\Lambda(A)}, \tag{35.16}$$

where $\Lambda(A)$ is the set of eigenvalues of A, V is a nonsingular matrix of eigenvectors (assuming A is diagonalizable), and $\|p_n\|_{\Lambda(A)}$ is defined by (35.14).

This theorem can be summarized in words as follows. If A is not too far from normal in the sense that $\kappa(V)$ is not too large, and if properly normalized degree n polynomials can be found whose size on the spectrum $\Lambda(A)$ decreases quickly with n, then GMRES converges quickly.

Example 35.1. Here is a numerical example. Let A be a 200×200 matrix whose entries are independent samples from the real normal distribution of mean 2 and standard deviation $0.5/\sqrt{200}$. In MATLAB,

```
m = 200; A = 2*eye(m) + 0.5*randn(m)/sqrt(m).
```
(35.17)

Figure 35.2 shows the eigenvalues of A, a set of points roughly uniformly distributed in the disk of radius $1/2$ centered at $z = 2$ (Exercise 12.3). Figure 35.3 shows the convergence curve for the GMRES iteration applied to the problem $Ax = b$, where $b = (1, 1, \ldots, 1)^*$. The convergence in this case is extraordinarily steady at a rate approximately 4^{-n}. The reason for this is not hard to spot. Since the spectrum of A approximately fills the disk indicated, $\|p(A)\|$ is approximately minimized by the choice $p(z) = (1 - z/2)^n$. Since $I - A/2$ is a random matrix scaled so that its spectrum approximately fills the disk of radius $1/4$ about 0, we have $\|p(A)\| = \|(I - A/2)^n\| \approx 4^{-n}$. This matrix A is well-conditioned, with condition number $\kappa(A) \approx 2.03$. The deviation from normality is modest, with $\kappa(V) \approx 141$.

Figure 35.3 illustrates the convergence of matrix iterations under favorable circumstances—when the matrix A is well behaved (which often means a good preconditioner has been applied). We see that six-digit accuracy is achieved after ten iterations, at a cost of approximately $10 \times 2m^2 = 8.0 \times 10^5$ flops, since the work is dominated by the matrix-vector multiplication at each step. Solving the same system by Gaussian elimination would require $\frac{2}{3}m^3 \approx 5.3 \times 10^6$ flops. This improvement by a factor close to 7 was achieved even though A has no sparsity to take advantage of and even though the dimension $m = 200$ is

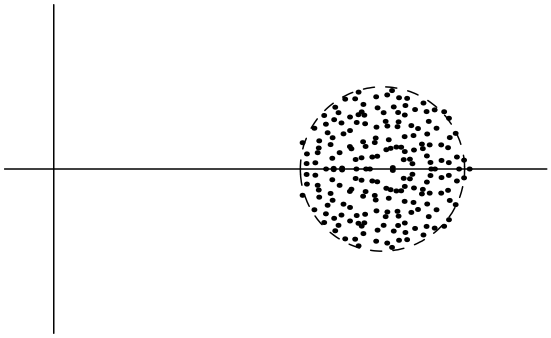


Figure 35.2. *Eigenvalues of the* 200×200 *matrix* A *of* (35.17). *The dashed curve is the circle of radius* $1/2$ *with center* $z = 2$ *in* $\mathbb{C}$. *The eigenvalues are approximately uniformly distributed within this disk.*

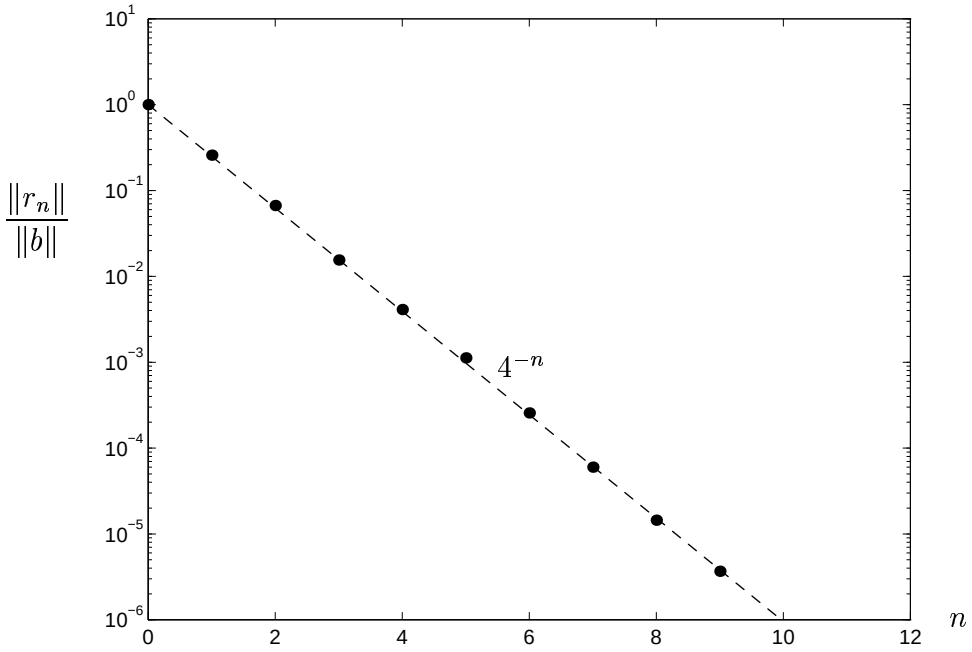


Figure 35.3. *GMRES convergence curve for the same matrix* A. *This rapid, steady convergence is illustrative of Krylov subspace iterations under ideal circumstances, when* A *is a well-behaved (or well-preconditioned) matrix.*

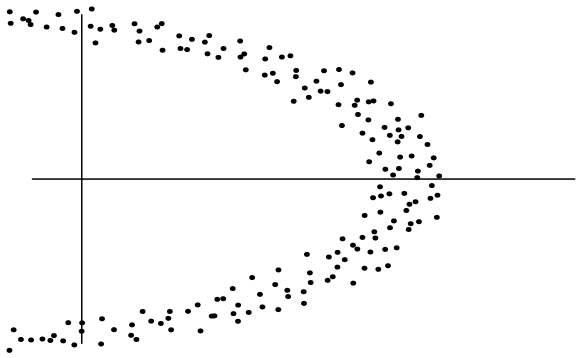


Figure 35.4. *Eigenvalues of a* 200×200 *matrix, like that of* (35.17) *except with a modified diagonal. Now the eigenvalues surround the origin on one side.*

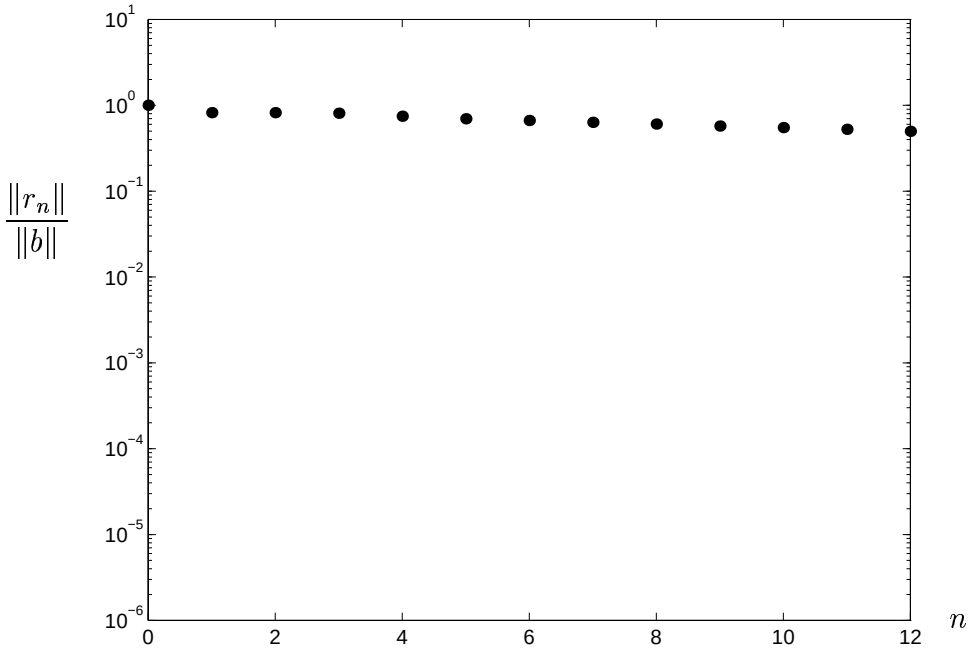


Figure 35.5. *GMRES convergence curve for the matrix of Figure* 35.4. *The convergence has slowed down greatly. When an iterative method stagnates like this, it is time to look for a better preconditioner.*

not high. For the same example with $m = 2000$, GMRES would beat Gaussian elimination by a factor more like 70. For a 2000×2000 matrix with similar spectral properties but 90% or 99% sparsity, the factor would improve to on the order of 700 or 7000, respectively, and the storage required by GMRES would also diminish dramatically. □

Example 35.2. If the eigenvalues of a matrix "surround the origin," on the other hand, such rapid convergence cannot be expected. Figures 35.4–35.5 present an example. The matrix is now $A' = A + D$, where A is the matrix of (35.17) and D is the diagonal matrix with complex entries

$$d_k = (-2 + 2\sin\theta_k) + i\cos\theta_k, \qquad \theta_k = \frac{k\pi}{m-1},\ 0 \le k \le m-1.$$

As is evident in Figure 35.4, the eigenvalues now lie in a semicircular cloud that bends around the origin. The convergence rate is much worse than before, making the iterative computation no better than Gaussian elimination for this problem. The condition numbers are now $\kappa(A) \approx 4.32$ and $\kappa(V) \approx 54.0$, so the deterioration in convergence cannot be explained by conditioning alone; it is the locations of the eigenvalues, not their magnitudes (or those of the singular values) that are causing the trouble. If the arc extended much further around the spectrum, the convergence would worsen further. □

Exercises

35.1. Show that if $S \subseteq \mathbb{C}$ contains infinitely many points, then (35.14) defines a norm on the vector space of all polynomials with complex coefficients. Explain what goes wrong if S has only finitely many points.

35.2. (a) Let $S \subseteq \mathbb{C}$ be a set whose convex hull contains 0 in its interior. That is, S is contained in no half-plane disjoint from the origin. Show that there is no $p \in P_1$ (i.e., no polynomial p of degree 1 with $p(0) = 1$) such that $\|p\|_S < 1$.

(b) Let A be a matrix, not necessarily normal, whose spectrum $\Lambda(A)$ has the property (a). Show that there is no $p \in P_1$ such that $\|p(A)\| < 1$.

(c) Though the convergence in Figure 35.5 is slow, it is clear that $\|r_1\| < \|r_0\|$. Explain why this does not contradict the result of (b). Describe what kind of polynomial $p_1 \in P_1$ GMRES has probably found to achieve $\|r_1\| < \|r_0\|$.

35.3. The recurrence $x_{n+1} = x_n + \alpha r_n = x_n + \alpha(b - Ax_n)$, where α is a scalar constant, is known as a *Richardson iteration*.

(a) What polynomial $p(A)$ at step n does this correspond to?

(b) What choice of α would you recommend for the matrix A of Figure 35.2, and what would you expect to be the corresponding convergence rate?

(c) Same questions for the matrix of Figure 35.4.

35.4. (a) Describe an $O(n^2)$ algorithm based on QR factorization by Givens rotations (Exercise 10.4) for solving the least squares problem of Algorithm 35.1.

(b) Show how the operation count can be improved to $O(n)$, as mentioned on p. 268, if the problem for step $n-1$ has already been solved.

35.5. Our statement of the GMRES algorithm (Algorithm 35.1) begins with the initial guess $x_0 = 0$, $r_0 = b$. (The same applies to CG and BCG, Algorithms 38.1 and 39.1.) Show that if one wishes to start with an arbitrary initial guess x_0, this can be accomplished by an easy modification of the right-hand side b.

35.6. For larger values of n, the cost of GMRES in operations and storage may be prohibitive. In such circumstances a method called *k-step restarted GMRES* or *GMRES(k)* is often employed, in which, after k steps, the GMRES iteration is started anew with the current vector x_k as an initial guess.

(a) Compare the asymptotic operation counts and storage requirements of GMRES and GMRES(k), for fixed k and increasing n.

(b) Describe an example in which GMRES(k) can be expected to converge in nearly as few iterations as GMRES (hence much faster in operation count).

(c) Describe another example in which GMRES(k) can be expected to fail to converge, whereas GMRES succeeds.

Lecture 36. The Lanczos Iteration

In the last three lectures we considered Krylov subspace iterations for nonhermitian matrix problems. We shall return to nonhermitian problems in Lecture 39, for there is more to this subject than Arnoldi and GMRES. But first, in this and the following two lectures, we specialize to the hermitian case, where a major simplification takes place.

Three-Term Recurrence

The Lanczos iteration is the Arnoldi iteration specialized to the case where A is hermitian. For simplicity of notation, we shall go a step further and assume, here and in the next two lectures, that A is real and symmetric.

Let us consider what happens to the Arnoldi process in this special case. Of course, all of the equations of Lectures 33 and 34 still apply, and in each formula we can replace * by ^{T}. The first thing we notice is that it follows from (33.12) that the Ritz matrix H_n is symmetric. Therefore its eigenvalues, the Ritz values or Lanczos estimates (33.10), are also real. This seems natural enough, since the eigenvalues of A are real.

The second thing we notice is more dramatic. Since H_n is both symmetric and Hessenberg, it is tridiagonal. This means that in the inner loop of the Arnoldi iteration (Algorithm 33.1), the limits 1 to n can be replaced by $n-1$ to n. Thus instead of the $(n+1)$-term recurrence (33.4) at step n, the Lanczos iteration involves just a three-term recurrence. The result is that each step of the Lanczos iteration is much cheaper than the corresponding step of the

Arnoldi iteration. In Lecture 38 we shall see that analogously, for solving $Ax = b$, each step of the conjugate gradient iteration is much cheaper than the corresponding step of GMRES.

The fact that H_n is tridiagonal is so important that it is worth reviewing how it arises from the symmetry of A. The key equation is (33.12), which we can write entry-wise for real matrices A, H_n, and Q_n as

$$h_{ij} = q_i^T A q_j. \tag{36.1}$$

This implies that $h_{ij} = 0$ for $i > j + 1$, since $Aq_j \in \langle q_1, q_2, ..., q_{j+1} \rangle$ and the Krylov vectors are orthogonal. Taking the transpose gives

$$h_{ij} = q_j^T A^T q_i. \tag{36.2}$$

If $A = A^T$, then $h_{ij} = 0$ for $j > i + 1$ by the same reasoning as before. This simple argument leading to a three-term recurrence relation applies to arbitrary self-adjoint operators, not just to matrices.

The Lanczos Iteration

Since a symmetric tridiagonal matrix contains only two distinct vectors, it is customary to replace the generic notation a_{ij} by new variables. Let us write $\alpha_n = h_{nn}$ and $\beta_n = h_{n+1,n} = h_{n,n+1}$. Then H_n becomes

$$T_n = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \beta_{n-1} \\ & & & \beta_{n-1} & \alpha_n \end{bmatrix}. \tag{36.3}$$

In this notation Algorithm 33.1 takes the following form.

Algorithm 36.1. Lanczos Iteration

$\beta_0 = 0$, $q_0 = 0$, $b =$ arbitrary, $q_1 = b/\|b\|$
for $n = 1, 2, 3, \ldots$
    $v = Aq_n$
    $\alpha_n = q_n^T v$
    $v = v - \beta_{n-1} q_{n-1} - \alpha_n q_n$
    $\beta_n = \|v\|$
    $q_{n+1} = v/\beta_n$

Each step consists of a matrix-vector multiplication, an inner product, and a couple of vector operations. If A has enough sparsity or other structure that matrix-vector products can be computed cheaply, then such an iteration can be applied without too much difficulty to problems of dimensions in the tens or hundreds of thousands.

The following theorem summarizes some of the properties of the Lanczos iteration (when carried out in exact arithmetic, of course, as with all such theorems in this book). Nothing here is new; these are restatements in the new notation of the results of Theorems 33.1 and 34.1 for the Arnoldi iteration.

Theorem 36.1. *The matrices Q_n of vectors q_n generated by the Lanczos iteration are reduced QR factors of the Krylov matrix* (33.6)*,*

$$K_n = Q_n R_n. \tag{36.4}$$

The tridiagonal matrices T_n are the corresponding projections

$$T_n = Q_n^* A Q_n, \tag{36.5}$$

and the successive iterates are related by the formula

$$A Q_n = Q_{n+1} \tilde{T}_n, \tag{36.6}$$

which we can write in the form of a three-term recurrence at step n,

$$A q_n = \beta_{n-1} q_{n-1} + \alpha_n q_n + \beta_n q_{n+1}. \tag{36.7}$$

As long as the Lanczos iteration does not break down (i.e., K_n is of full rank n), the characteristic polynomial of T_n is the unique polynomial $p^n \in P^n$ that solves the Arnoldi/Lanczos approximation problem (34.3)*, i.e., that achieves*

$$\| p^n(A) b \| = \textit{minimum}. \tag{36.8}$$

Lanczos and Electric Charge Distributions

In practice, the Lanczos iteration is used to compute eigenvalues of large symmetric matrices just as the Arnoldi iteration is used for nonsymmetric matrices (Lecture 34). At each step n, or at occasional steps, the eigenvalues of the growing tridiagonal matrix T_n are determined by standard methods. These are the Ritz values or "Lanczos estimates" (33.10) for the given matrix A and starting vector q_1. Often some of these numbers are observed to converge geometrically to certain limits, which can then be expected to be eigenvalues of A.

As with the Arnoldi iteration, it is the outlying eigenvalues of A that are most often obtained first. This assertion can be made more precise by the following rule of thumb:

If the eigenvalues of A are more evenly spaced than Chebyshev points, then the Lanczos iteration will tend to find outliers.

Here is what this statement means. Suppose the m eigenvalues $\{\lambda_j\}$ of A are spread reasonably densely around an interval on the real axis. Since the Lanczos iteration is scale- and translation-invariant (Theorem 34.2), we can assume without loss of generality that this interval is $[-1, 1]$. The m *Chebyshev points* in $[-1, 1]$ are defined by the formula

$$x_j = \cos\theta_j, \qquad \theta_j = \frac{(j - \frac{1}{2})\pi}{m}, \qquad 1 \le j \le m. \tag{36.9}$$

The exact definition is not important; what matters is that these points cluster quadratically near the endpoints, with the spacing between points $O(m^{-1})$ in the interior and $O(m^{-2})$ near ± 1. The rule of thumb asserts that if the eigenvalues $\{\lambda_j\}$ of A are more evenly distributed than this—less clustered at the endpoints—then the Ritz values computed by a Lanczos iteration will tend to converge to the outlying eigenvalues first. In particular, an approximately uniform eigenvalue distribution will produce rapid convergence towards outliers. Conversely, if the eigenvalues of A are more than quadratically clustered at the endpoints—a situation not so common in practice—then we can expect convergence to some of the "inliers."

These observations can be given a physical interpretation. Consider m point charges free to move about the interval $[-1, 1]$. Assume that the repulsive force between charges located at x_j and x_k is proportional to $|x_j - x_k|^{-1}$. (For electric charges in 3D the force would be $|x_j - x_k|^{-2}$, but this becomes $|x_j - x_k|^{-1}$ in 2D, where we can view each point as the intersection of an infinite line in 3D with the plane.) Let these charges distribute themselves in a minimal-energy equilibrium in $[-1, 1]$. Then this minimal-energy distribution and the Chebyshev distribution are approximately the same, and in the limit $m \to \infty$, they both converge to a limiting continuous charge density distribution proportional to $(1 - x^2)^{-1/2}$.

Think of the eigenvalues of A as point charges. If they are distributed approximately in a minimal-energy configuration in an interval, then the Lanczos iteration will be useless; there will be little convergence before step $n = m$. If the distribution is very different from this, however, then there is likely to be rapid convergence to some eigenvalues, namely, the eigenvalues in regions where there is "too little charge" in the sense that if the points were free to move, more would tend to cluster here. The rule of thumb can now be restated:

The Lanczos iteration tends to converge to eigenvalues in regions of "too little charge" for an equilibrium distribution.

The explanation of this observation depends on the connection (36.8) of the Lanczos iteration with polynomial approximation. Some of the details are worked out in Exercise 36.2.

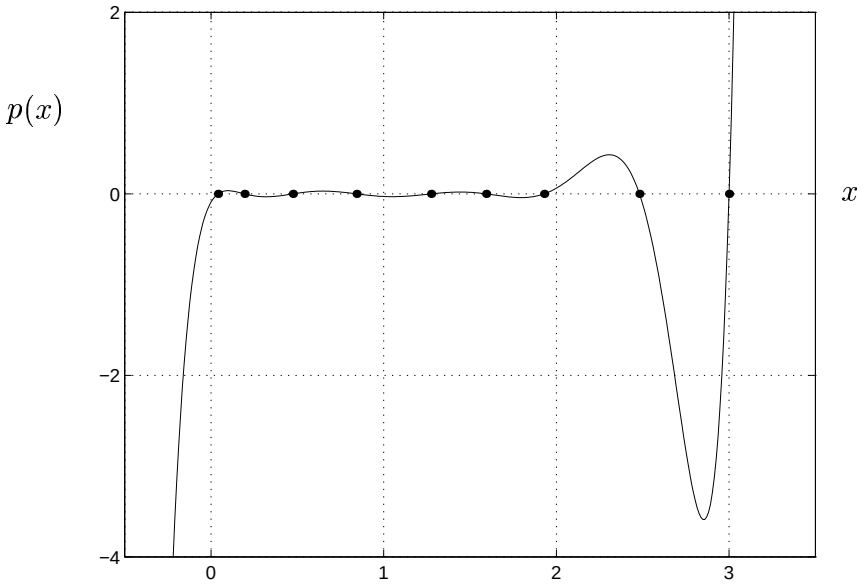


Figure 36.1. *Plot of the Lanczos polynomial at step* 9 *of the Lanczos iteration for the matrix* (36.10). *The roots are the Ritz values or "Lanczos eigenvalue estimates." The polynomial is small throughout* $[0, 2] \cup \{2.5\} \cup \{3.0\}$. *To achieve this, it must place one root near* 2.5 *and another very near* 3.0.

Example

The convergence of the Lanczos iteration is best illustrated by a numerical example. Let A be the 203×203 matrix

$$A = \text{diag}(0, .01, .02, \ldots, 1.99, 2, \quad 2.5, \quad 3.0). \tag{36.10}$$

The spectrum of A consists of a dense collection of eigenvalues throughout $[0, 2]$ together with two outliers, 2.5 and 3.0. We carry out a Lanczos iteration beginning with a random starting vector q_1.

Figure 36.1 shows the Ritz values and the associated Lanczos polynomial at step $n = 9$. Seven of the Ritz values lie in $[0, 2]$, and the polynomial is uniformly small on that interval; the beginnings of a tendency for the Ritz values to cluster near the endpoints can be detected. The other two Ritz values lie near the eigenvalues at 2.5 and 3.0. The leading three Ritz values are

$$1.93, \quad 2.48, \quad 2.999962.$$

Evidently we have little accuracy in the lower eigenvalues but five-digit accuracy in the leading one. A plot like this gives an idea of why outliers tend to be estimated accurately. The graph of $p(x)$ is so steep for $x \approx 3$ that if $p(3)$ is to be small, there must be a root of p very close to 3. This steepness of the graph is related to the presence of "too little charge" near this point. If the

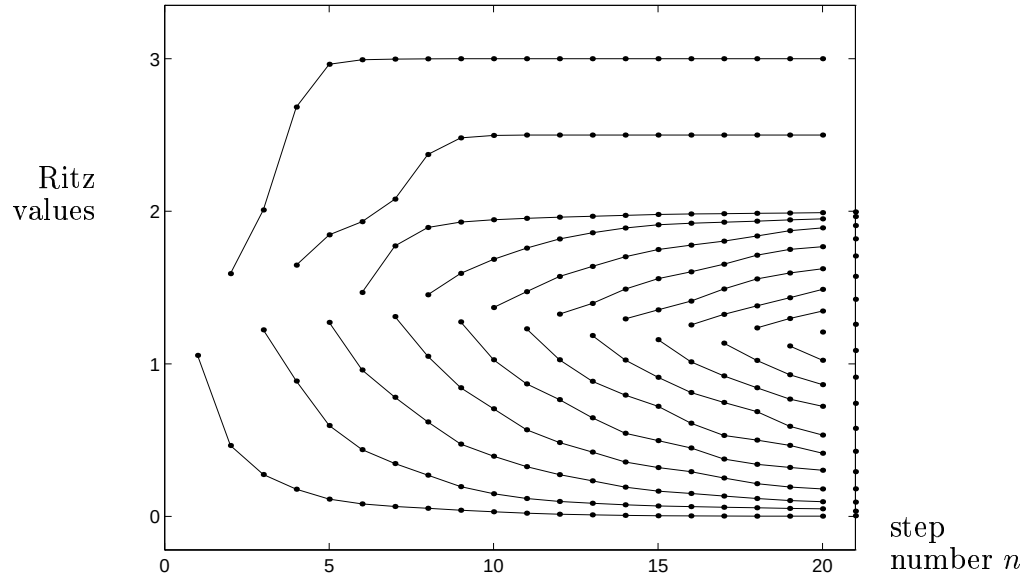


Figure 36.2. *Ritz values for the first* 20 *steps of the Lanczos iteration applied to the same matrix. The convergence to the eigenvalues* 2.5 *and* 3.0 *is geometric. Little useful convergence to individual eigenvalues occurs in the* $[0,2]$ *part of the spectrum. Instead, the Ritz values in* $[0,2]$ *approximate Chebyshev points in that interval, marked by dots on the right-hand boundary.*

charges were free to move about $[0,3]$ to minimize energy, more points would cluster near $x = 3$, and $p(x)$ would not be so steep there.

At step 20 the leading three Ritz values are

$$1.9906, \quad 2.499999999987, \quad 3.00000000000000.$$

Now we have about fifteen digits of accuracy in the leading eigenvalue and twelve digits in the second. A plot of $p(x)$ would be correspondingly steep near the points 2.5 and 3.0. Note that convergence to the third eigenvalue is also beginning to occur, a reflection of the fact that the eigenvalues in $[0,2]$ are distributed evenly rather than in a Chebyshev distribution.

An "aerial view" of the convergence process appears in Figure 36.2, which shows the Ritz values for all steps from $n = 1$ to $n = 20$. Each vertical slice of this plot corresponds to the Ritz values at one iteration; the lines connecting the dots help the eye follow what is going on but have no precise meaning. The plot shows pronounced convergence to the leading eigenvalue after about $n = 5$ and to the next one around $n = 10$. In the interval $[0,2]$ containing the other eigenvalues, they show a density of Ritz values approximately proportional to $(1 - x^2)^{-1/2}$, with very clear bunching at endpoints.

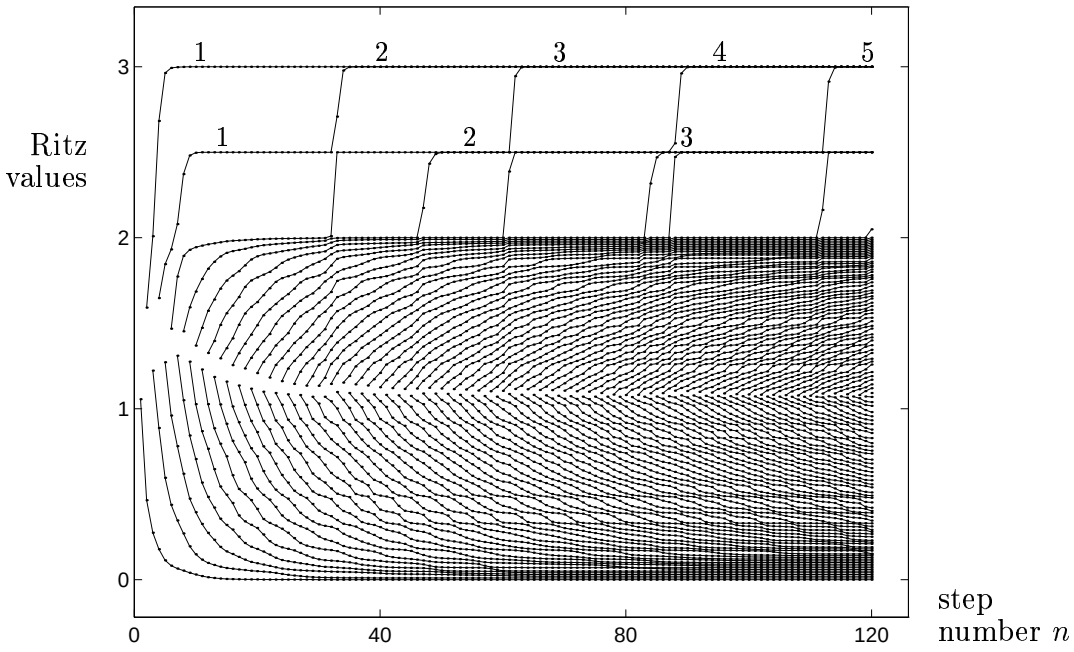


Figure 36.3. *Continuation to* 120 *steps of the Lanczos iteration. The numbers indicate multiplicities of the Ritz values. Note the appearance of four "ghost" copies of the eigenvalue* 3.0 *and two "ghost" copies of the eigenvalue* 2.5.

Rounding Errors and "Ghost" Eigenvalues

Rounding errors have a complex effect on the Lanczos iteration and, indeed, on all iterations of numerical linear algebra based on three-term recurrence relations. The source of the difficulty is easily identified. In an iteration based on an n-term recurrence relation, such as Arnoldi or GMRES, the vectors $q_1, q_2, q_3, \ldots$ are forced to be orthogonal by explicit Gram–Schmidt operations. Three-term recurrences like Lanczos and conjugate gradients, however, depend upon orthogonality of the vectors $\{q_j\}$ to arise "automatically" from a mathematical identity. In practice, such identities are not accurately preserved in the presence of rounding errors, and after a number of iterations, orthogonality is lost.

The loss of orthogonality in practical Lanczos iterations sounds wholly bad, but the situation is more subtle than that. As it happens, loss of orthogonality is connected closely with the convergence of Ritz values to eigenvalues of A. A great deal is known about this subject, though not as much as one might like; we shall not give details.

Because of complexities like these, no straightforward theorem is known to the effect that the Lanczos or conjugate gradient iterations is stable in the sense defined in this book. Nonetheless, these iterations are extraordinarily useful in practice. Figure 36.3 gives an idea of the way in which instability

is often manifested in practice without preventing the iteration from being useful. The figure is a repetition of Figure 36.2, but for 120 instead of 20 steps of the iteration. Everything looks as expected until around step 30, when a second copy of the eigenvalue 3.0 appears among the Ritz values. A third copy appears around step 60, a fourth copy around step 90, and so on. Meanwhile, additional copies of the eigenvalue 2.5 also appear around step 40 and 80 and (just beginning to be visible) 120. These extra Ritz values are known as "ghost" eigenvalues, and they have nothing to do with the actual multiplicities of the corresponding eigenvalues of A.

A rigorous analysis of the phenomenon of ghost eigenvalues is complicated. Intuitive explanations, however, are not hard to devise. One idea is that in the presence of rounding errors, one should think of each eigenvalue of A not as a point but as a small interval of size roughly $O(\epsilon_{\text{machine}}\|A\|)$; ghost eigenvalues arise from the need for $p(z)$ to be small not just at the exact eigenvalues but throughout these small intervals. Another, rather different explanation is that convergence of a Ritz value to an eigenvalue of A annihilates the corresponding eigenvector component in the vector being operated upon; but in the presence of rounding errors, random noise must be expected to excite that component slightly again. After sufficiently many iterations, this previously annihilated component will have been amplified enough that another Ritz value is needed to annihilate it again—and then again, and again.

Both of these explanations capture some of the truth about the behavior of the Lanczos iteration in floating point arithmetic. The second one has perhaps more quantitative accuracy.

Exercises

36.1. In Lecture 27 it was pointed out that the eigenvalues of a symmetric matrix $A \in \mathbb{R}^{m\times m}$ are the stationary values of the Rayleigh quotient $r(x) = (x^TAx)/(x^Tx)$ for $x \in \mathbb{R}^m$. Show that the Ritz values at step n of the Lanczos iteration are the stationary values of $r(x)$ if x is restricted to $\mathcal{K}_n$.

36.2. Consider a polynomial $p \in P^n$, i.e., $p(z) = \prod_{k=1}^n (z - z_k)$ for some $z_k \in \mathbb{C}$.

(a) Write $\log|p(z)|$ as a sum of n terms corresponding to the points z_k.

(b) Explain why the term involving z_k can be interpreted as the potential corresponding to a negative unit point charge located at z_k, if charges repel in inverse proportion to their separation. Thus $\log|p(z)|$ can be viewed as the potential at z induced by n point charges.

(c) Replacing each charge -1 by $-1/n$ and taking the limit $n \to \infty$, we get a continuous charge density distribution $\mu(\zeta)$ with integral -1, which we can expect to be related to the limiting density of zeros of polynomials $p \in P^n$ as

$n \to \infty$. Write an integral representing the potential $\varphi(z)$ corresponding to $\mu(\zeta)$, and explain its connection to $|p(z)|$.

(d) Let S be a closed, bounded subset of $\mathbb{C}$ with no isolated points. Suppose we seek a distribution $\mu(z)$ with support in S that minimizes $\max_{z \in S} \varphi(z)$. Give an argument (not rigorous) for why such a $\mu(z)$ should satisfy $\varphi(z) =$ constant throughout S. Explain why this means that the "charges" are in equilibrium, experiencing no net forces. In other words, S is like a 2D electrical conductor on which a quantity -1 of charge has distributed itself freely. Except for an additive constant, $\varphi(z)$ is the *Green's function* for S.

(e) As a step toward explaining the rule of thumb of p. 279, suppose that A is a real symmetric matrix with spectrum densely distributed in $[a,b] \cup \{c\} \cup [d,e]$ for $a < b < c < d < e$. Thus (b,d) is a region of "too little charge" for the set $S = [a,e]$. Explain why rapid convergence of a Ritz value to c can be expected, and estimate the rate of convergence in terms of the equilibrium potential $\varphi(z)$ associated with the set $S' = [a,b] \cup [c,d]$.

36.3. Let A be the 1000×1000 symmetric matrix whose entries are all zero except for $a_{ij} = \sqrt{i}$ on the diagonal, $a_{ij} = 1$ on the sub- and superdiagonals, and $a_{ij} = 1$ on the 100th sub- and superdiagonals, i.e., for $|i-j| = 100$. Determine the smallest eigenvalue of A to six digits of accuracy by the Lanczos iteration.

36.4. As a special case of the Arnoldi lemniscates of Lecture 34, "Lanczos lemniscates" can be employed to illustrate the convergence of the Lanczos iteration. Find a way to modify your program of Exercise 36.3 to plot the Lanczos lemniscates at each step. Your method need not be elegant, efficient, or numerically robust. Produce plots of Lanczos lemniscates at steps $n = 1, 2, \ldots, 12$ for the example of Figure 36.2 and for an example of your own choosing.

Lecture 37. From Lanczos to Gauss Quadrature

If discrete vectors become continuous functions on $[-1,1]$, and the matrix A is taken to be the operator of pointwise multiplication by x, then the Lanczos iteration becomes the standard procedure for constructing orthogonal polynomials via a three-term recurrence relation. From here it is a short step to Gauss quadrature formulas, whose nodes and weights can be computed by solving a symmetric tridiagonal matrix eigenvalue problem.

Orthogonal Polynomials

In Lecture 7 we considered a continuous analogue of QR factorization. We now consider a continuous analogue of the Lanczos iteration, restricting attention, as in the last lecture, to real vectors (now functions) and real symmetric matrices (now linear operators).

The first thing we do is replace $\mathbb{R}^m$ by $L^2[-1,1]$, a vector space of real-valued functions on $[-1,1]$. The inner product of two functions $u, v \in L^2[-1,1]$ is defined by

$$(u,v) = \int_{-1}^{1} u(x)\, v(x)\, dx, \tag{37.1}$$

and the norm of a function $u \in L^2[-1,1]$ is $\|u\| = (u,u)^{1/2}$.

In Lecture 7 we took A to be a "$[-1,1] \times n$ matrix" whose columns were powers of x. Here, for the Lanczos iteration, A should be square instead of rectangular. We shall take it to be the "$[-1,1] \times [-1,1]$ matrix" corresponding

to pointwise multiplication by x. That is, A is the linear operator defined by the equation

$$(Au)(x) \;=\; xu(x) \tag{37.2}$$

for each $u \in L^2[-1,1]$. This operator is analogous to a diagonal matrix, with the continuum of values $x \in [-1,1]$ along the "diagonal." (One can make the diagonal matrix idea precise by describing A as an integral operator with kernel $k(x,y) = x\delta(x-y)$, where δ is the Dirac delta function.) In particular, A is symmetric, and because of the symmetry, the Arnoldi process specializes to the Lanczos process.

There is one more item to be pinned down. Let us agree that the initial function $b(x)$ will be a nonzero constant. The corresponding normalized initial function $q_1(x)$ will accordingly be $q_1(x) = 1/\sqrt{2}$.

The Lanczos iteration (Algorithm 36.1) now takes the following form.

Algorithm 37.1. Construction of Orthogonal Polynomials

$\beta_0 = 0, \;\; q_0(x) = 0, \;\; q_1(x) = 1/\sqrt{2}$
for $n = 1, 2, 3, \ldots$
$\quad v(x) = xq_n(x)$
$\quad \alpha_n = (q_n, v)$
$\quad v(x) = v(x) - \beta_{n-1}q_{n-1}(x) - \alpha_n q_n(x)$
$\quad \beta_n = \|v\|$
$\quad q_{n+1}(x) = v(x)/\beta_n$

Note that the polynomials $q_1, q_2, q_3, \ldots$ are of degrees $0, 1, 2, \ldots$, and thus the polynomial of degree n in this sequence is q_{n+1}.

Algorithm 37.1 can be found in many books that have nothing ostensibly to do with linear algebra. It is precisely the usual three-term recurrence for constructing the sequence of *Legendre polynomials*, orthogonal on the interval $[-1,1]$. Our statement of the algorithm is unusual, however, in notation and in normalization. The nth Legendre polynomial is usually written $P_n(x)$, of degree n, and normalized by $P_n(1) = 1$. Thus our $q_{n+1}(x)$ is a scalar multiple of the usual $P_n(x)$. Nevertheless we shall call it a Legendre polynomial.

The Legendre polynomials were considered already in (7.11) and Figure 7.1, where they were derived by Gram–Schmidt factorization of the "$[-1,1] \times n$ matrix" of monomials mentioned above. That matrix had the form of the Krylov matrix (33.6) generated by our present initial vector b and operator A, and that is why the Gram–Schmidt process in Lecture 7 had the same effect as the Lanczos process here.

We labeled Algorithm 37.1 "Construction of orthogonal polynomials" rather than "Construction of Legendre polynomials" because it is, in fact, more

general. If (37.1) is modified by the inclusion of a nonconstant positive weight function $w(x)$ in the integrand, then one obtains other families of orthogonal polynomials such as Chebyshev polynomials and Jacobi polynomials. All of the developments of this lecture apply to these more general families, but we shall not give details.

Jacobi Matrices

All the formulas of the foregoing lectures on Arnoldi and Lanczos iterations remain valid for the polynomial orthogonalization process we have just described. Of course, they have to be interpreted properly, with $1, 2, \ldots, m$ replaced by $[-1, 1]$ and the usual vector inner product replaced by (37.1). For example, we now have "$[-1, 1] \times n$ matrices"

$$K_n = \left[\begin{array}{c|c|c|c} 1 & x & \cdots & x^{n-1} \end{array}\right], \qquad Q_n = \left[\begin{array}{c|c|c|c} q_1(x) & q_2(x) & \cdots & q_n(x) \end{array}\right], \tag{37.3}$$

and they are related to one another exactly as in (36.4). (As just mentioned, K_n is the matrix that was called A in Lecture 7.)

The tridiagonal matrices $\{T_n\}$ described in the previous lecture are particularly important. There are still $n \times n$ discrete matrices, related to Q_n and A by (36.5) and (36.6). Their entries are given by the analogue of (36.1),

$$t_{ij} \;=\; (q_i(x), xq_j(x)). \tag{37.4}$$

In the context of orthogonal polynomials, the matrices $\{T_n\}$ are known as *Jacobi matrices*. The three-term recurrence (36.7) takes the form

$$xq_n(x) = \beta_{n-1}q_{n-1}(x) + \alpha_n q_n(x) + \beta_n q_{n+1}(x). \tag{37.5}$$

The statement of Algorithm 37.1 is perhaps misleading as written. It would appear that nontrivial computations are involved at each step of this algorithm: the evaluation of the inner product (q_n, v) and norm $\|v\|$ that define α_n and β_n. If (37.1) contained an arbitrary weight function $w(x)$, these computations would indeed be nontrivial. However, for the particular choice $w(x) = 1$ associated with Legendre polynomials, and also for various choices associated with other classical families of polynomials, the entries $\{\alpha_n\}$ and $\{\beta_n\}$ are known analytically. In the notation of (36.3), we have

$$\alpha_n = 0, \qquad \beta_n = \frac{1}{2}(1 - (2n)^{-2})^{-1/2} \tag{37.6}$$

for Legendre polynomials. With the use of these formulas Algorithm 37.1 becomes a trivial mechanical procedure, a three-term recurrence relation and nothing more.

The Characteristic Polynomial

What becomes of the Arnoldi/Lanczos approximation problem (36.8) in this context of orthogonal polynomials? The answer comes upon noting that for our special choices of b and A, we have $p(A)\,b = p(x)/\sqrt{2}$. It follows that (36.8) can be written in the following way.

Orthogonal Polynomials Approximation Problem. Find $p^n \in P^n$ such that

$$\| p^n(x) \| = \text{minimum}. \tag{37.7}$$

According to Theorem 36.1, the solution is the characteristic polynomial of the matrix T_n.

From here it is a short step to a remarkable conclusion. Note that any $p \in P^n$ can be written in the form $p(x) = Cq_{n+1}(x) + Q_n y$, where C is a constant—the inverse of the leading coefficient of $q_{n+1}(x)$. Note also that since the functions $\{q_n(x)\}$ are orthogonal, we have $\|p\| = (C^2 + \|y\|^2)^{1/2}$. It follows that the minimum in (37.7) is achieved by setting $y = 0$. In other words, $p^n(x)$ *is the same as* $q_{n+1}(x)$ up to that constant C. We express this conclusion as a theorem, the final assertion of which is proved in Exercise 37.2.

Theorem 37.1. *Let $\{q_n(x)\}$ be the sequence of orthogonal polynomials generated by Algorithm* 37.1*, let $\{T_n\}$ be the associated sequence of tridiagonal Jacobi matrices, and let p^n be the characteristic polynomial of T_n. Then for $n = 0, 1, 2, \ldots,$*

$$p^n(x) \;=\; C_n q_{n+1}(x), \tag{37.8}$$

where C_n is a constant. In particular, the zeros of $q_{n+1}(x)$ are the eigenvalues of T_n. These n zeros are distinct and lie in the open interval $(-1, 1)$.

This theorem is of great computational importance. To determine the zeros of the Legendre polynomials, all one has to do is compute the eigenvalues of the associated Jacobi matrices, whose entries are given in closed form by (37.6). As we have seen in previous lectures, the eigenvalue problem for an $n \times n$ symmetric tridiagonal matrix is well-conditioned and can be solved very quickly, requiring only $O(n^2)$ flops. By contrast, computing the zeros of the Legendre polynomials directly, starting from the coefficients of the polynomials rather than the Jacobi matrices, is inefficient and numerically unstable.

Quadrature Formulas

There is a reason why the zeros of the Legendre polynomials are of computational interest: they are the nodes of the Gauss–Legendre quadrature formulas.

Let us briefly review the idea of numerical quadrature. Suppose $f(x)$ is a function defined on $[-1,1]$ and we want to compute the integral

$$I(f) \;=\; \int_{-1}^{1} f(x)\,dx. \tag{37.9}$$

(If an interval of integration other than $[-1,1]$ is of interest, this can be handled by a linear change of variables.) It is natural to consider approximating $I(f)$ by a finite sum

$$I_n(f) \;=\; \sum_{j=1}^{n} w_j f(x_j) \tag{37.10}$$

defined by a set of n *nodes* or *abscissas* $x_j \in [-1,1]$ and corresponding *weights* w_j, chosen independently of f. This is an n-point *quadrature formula*, a notion studied by numerical analysts going back to Newton. Various forms of such formulas, often coupled with adaptive error estimation, interval subdivision, and order control, are the basis of most numerical integration carried out on computers today.

Any set of nodes is a candidate for a quadrature formula. The following result is a consequence of the nonsingularity of Vandermonde matrices (Exercise 37.3).

Theorem 37.2. *Let the nodes $\{x_j\}$ be an arbitrary set of n distinct points in $[-1,1]$. Then there is a unique choice of weights $\{w_j\}$ with the property that the quadrature formula* (37.10) *has order of accuracy at least $n-1$ in the sense that it is exact if $f(x)$ is any polynomial of degree $\le n-1$.*

If the nodes $\{x_j\}$ are taken equally spaced from -1 to 1, the quadrature formula provided by this theorem is known as a *Newton–Cotes* formula, the n-point generalization of the familiar trapezoid and Simpson rules. Newton–Cotes formulas have the order of accuracy guaranteed by this theorem but no higher. These formulas are simple and useful, especially for lower values of n. For larger n, their weights w_j have oscillating signs and huge amplitudes, of order 2^n, causing numerical instability.

Gauss Quadrature

The idea of Gauss quadrature is to pick not just the weights $\{w_j\}$ but also the nodes $\{x_j\}$ optimally, so as to raise the order of accuracy of (37.10) as high as possible. As it happens, there is a unique choice of nodes and weights that achieves this, and the resulting formula has order $2n-1$. This is a dramatic improvement over order $n-1$, a doubling of the number of digits of accuracy typically attainable for smooth functions and a fixed number of function evaluations. Moreover, the weights w_j are all positive, making these formulas stable even for high n.

What is the magic set of nodes $x_1, \ldots, x_n$ that doubles the order of accuracy of an n-point quadrature formula? It is nothing more than the set of zeros of the Legendre polynomial $q_{n+1}(x)$. The *Gauss* or *Gauss–Legendre quadrature formula* is defined as the quadrature formula (37.10) provided by Theorem 37.2 whose nodes $x_1, \ldots, x_n$ are the zeros of $q_{n+1}(x)$.

Theorem 37.3. *The n-point Gauss–Legendre quadrature formula has order of accuracy exactly $2n-1$, and no quadrature formula* (37.10) *has order of accuracy higher than this.*

Proof. Given any set of distinct points $\{x_j\}$, let $f(x)$ be the polynomial $\prod_{j=1}^{n}(x-x_j)^2$ of degree $2n$. Then $I(f) > 0$, but $I_n(f) = 0$ since $f(x_j) = 0$ for each node x_j. Thus the quadrature formula is not exact for polynomials of degree $2n$.

On the other hand, suppose $f(x)$ is any polynomial of degree $\leq 2n-1$, and take $\{x_j\}$ to be the Gauss quadrature nodes, the zeros of $\{q_{n+1}(x)\}$. The function $f(x)$ can be factored in the form

$$f(x) \;=\; g(x)q_{n+1}(x) + r(x),$$

where $g(x)$ is a polynomial of degree $\leq n-1$ and $r(x)$, the remainder term, is also a polynomial of degree $\leq n-1$. (In fact, $r(x)$ is the degree $n-1$ polynomial interpolant to f in the points $\{x_j\}$.) Now since $q_{n+1}(x)$ is orthogonal to all polynomials of lower degree, we have $I(gq_{n+1}) = 0$. At the same time, since $g(x_j)q_{n+1}(x_j) = 0$ for each node x_j, we have $I_n(gq_{n+1}) = 0$. Since I and I_n are linear operators, these identities imply $I(f) = I(r)$ and $I_n(f) = I_n(r)$. But since $r(x)$ is of degree $\leq n-1$, we have $I(r) = I_n(r)$ by Theorem 37.2, and combining these results gives $I(f) = I_n(f)$, as claimed. □

Gauss Quadrature via Jacobi Matrices

In six pages we have gone from the Lanczos iteration to Legendre polynomials and from Legendre polynomials to Gauss quadrature. We have even provided a fast and stable algorithm for determining the nodes of Gauss quadrature formulas: just set up the Jacobi matrices $\{T_n\}$ and compute their eigenvalues.

One final observation will finish the story. Not only the nodes but also the weights of Gauss quadrature formulas can be obtained from the eigenvalue problem for T_n. The jth Gauss weight turns out to be simply $w_j = 2(v_j)_1^2$, that is, twice the square of the first component of the jth eigenvector of T_n. We state this result without proof. An analogous result holds for Gauss quadrature formulas defined by inner products (37.1) with general weight functions $w(x)$.

Theorem 37.4. *Let T_n be the $n \times n$ Jacobi matrix* (36.3) *defined by Algorithm* 37.1 *or* (36.5)*, with entries $\beta_1, \ldots, \beta_{n-1}$ given by* (37.6)*. Let $T_n = VDV^T$ be an orthogonal diagonalization of T_n with $V = [v_1|\cdots|v_n]$ and $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. Then the nodes and weights of the Gauss–Legendre quadrature formula are given by*

$$x_j = \lambda_j, \qquad w_j = 2(v_j)_1^2, \qquad j = 1, \ldots, n. \tag{37.11}$$

Example

As an illustration of the power of Gauss quadrature for integrating smooth functions, suppose we wish to evaluate the integral

$$I(e^x) \;=\; \int_{-1}^{1} e^x \, dx \;=\; 2.35040239.$$

Taking $n = 4$, we find that the Jacobi matrix for four-point Gauss–Legendre quadrature is

$$T_4 \;=\; \begin{bmatrix} 0 & 0.577350269 & & \\ 0.577350269 & 0 & 0.516397779 & \\ & 0.516397779 & 0 & 0.507092553 \\ & & 0.507092553 & 0 \end{bmatrix}.$$

The eigenvalues of this matrix give the nodes

$$x_1 = -x_4 = 0.861136312, \qquad x_2 = -x_3 = 0.339981044,$$

and the first components of the eigenvectors give the corresponding weights

$$w_1 = w_4 = 0.347854845, \qquad w_2 = w_3 = 0.652145155.$$

Evaluating the sum (37.10) gives

$$I_n(e^x) = 2.35040209,$$

which agrees with the exact result to about seven digits. The four-point Newton–Cotes formula, by contrast, gives $I_n(e^x) \approx 2.3556$, accurate to only about three digits.

Exercises

37.1. The standard recurrence relation for Legendre polynomials is

$$P_n(x) \;=\; \frac{2n-1}{n} x P_{n-1}(x) - \frac{n-1}{n} P_{n-2}(x) \tag{37.12}$$

with initial values $P_0(x) = 1$, $P_1(x) = x$.

(a) Confirm that (37.12) gives the polynomials $P_2(x)$ and $P_3(x)$ of (7.11).

(b) Since $\{P_n(x)\}$ and $\{q_{n+1}(x)\}$ are normalized differently, (37.12) is not the same as the recurrence (37.5) with coefficients (37.6). Write down the two tridiagonal matrices corresponding to these formulas, and derive the relationship between them.

(c) Use the result of (b) to determine a formula for $q_{n+1}(1)$, or equivalently, for $\|P_n\|$.

37.2. Show based on the definition of orthogonality that $q_{n+1}(x)$ has n distinct zeros, all contained in the open interval $(-1, 1)$. (The fact that they are distinct also follows from Exercise 25.1, but here, use a direct argument.)

37.3. The problem of interpolating n data values $\{y_j\}$ in n distinct data points $\{x_j\}$ by a polynomial of degree $\le n - 1$ was expressed in (11.4) as a square Vandermonde linear system of equations.

(a) Prove that this Vandermonde matrix is nonsingular by arguing that if the interpolation problem has a solution, it must be unique.

(b) Write down the analogous system of equations implicit in Theorem 37.2. Using the result of (a), prove this theorem.

37.4. (a) Write a six-line MATLAB program that computes the nodes and weights for the n-point Gauss–Legendre quadrature formula and applies these numbers to compute the approximate integral of the function f.

(b) Taking $f(x) = e^x$ and $n = 4$, confirm the example in the text. Then plot $|I(e^x) - I_n(e^x)|$ on a log scale for $n = 1, 2, \ldots, 40$ and comment on the results.

(c) Produce a similar plot for $f(x) = e^{|x|}$, and comment.

37.5. The program of Exercise 37.4 computes the zeros of Legendre polynomials, also known as *Legendre points* in $[-1, 1]$. The zeros of Chebyshev polynomials, *Chebyshev points*, are given by the explicit formula (36.9). Perform a sequence of calculations to generate numbers and plots illustrating as elegantly as you can that in the limit $n \to \infty$, both Legendre and Chebyshev points approach the limiting density distribution $\mu(x) = \pi^{-1}(1 - x^2)^{-1/2}$ (in the notation of Exercise 36.2). Produce further plots and numbers to explore the question: how close are Legendre points to Chebyshev points for various values of n?

Lecture 38. Conjugate Gradients

The conjugate gradient iteration is the "original" Krylov subspace iteration, the most famous of these methods and one of the mainstays of scientific computing. Discovered by Hestenes and Stiefel in 1952, it solves symmetric positive definite systems of equations amazingly quickly if the eigenvalues are well distributed.

Minimizing the 2-Norm of the Residual

As in the last two lectures, let $A \in \mathbb{R}^{m \times m}$ be real and symmetric, and suppose we wish to solve a nonsingular system of equations $Ax = b$, with exact solution $x_* = A^{-1}b$. Let $\mathcal{K}_n$ denote the nth Krylov subspace (33.5) generated by b,

$$\mathcal{K}_n \;=\; \langle b, Ab, \ldots, A^{n-1}b \rangle. \tag{38.1}$$

One approach based on this Krylov subspace would be to solve the system by GMRES. As described in Lecture 35, this would mean that at step n, x_* is approximated by the vector $x_n \in \mathcal{K}_n$ that minimizes $\|r_n\|_2$, where $r_n = b - Ax_n$. Actually, the usual GMRES algorithm does more work than is necessary for minimizing $\|r_n\|_2$. Since A is symmetric, faster algorithms are available based on three-term instead of $(n+1)$-term recurrences at step n. One of these goes by the names of conjugate residuals or MINRES ("minimal residuals").

These methods, at least when constructed to apply to both definite and indefinite matrices, involve certain complications. Rather than describe them, we turn directly to the simpler and more important positive definite case.

Minimizing the A-Norm of the Error

Assume that A is not only real and symmetric but also *positive definite*. As discussed in Lecture 23, this means that the eigenvalues of A are all positive, or equivalently, that $x^TAx > 0$ for every nonzero $x \in \mathbb{R}^m$. Under this assumption, the function $\|\cdot\|_A$ defined by

$$\|x\|_A = \sqrt{x^TAx} \tag{38.2}$$

is a norm on $\mathbb{R}^m$, as can be verified from the definition (3.1). It is called the *A-norm*. (This is the same as the norm $\|x\|_W$ of (3.3), if W is a Cholesky factor of A or any other matrix satisfying $W^TW = A$.)

The vector whose A-norm will concern us is $e_n = x_* - x_n$, the error at step n. The conjugate gradient iteration can be described as follows. *It is a system of recurrence formulas that generates the unique sequence of iterates $\{x_n \in \mathcal{K}_n\}$ with the property that at step n, $\|e_n\|_A$ is minimized.*

We shall present the formulas for the CG iteration, without motivation at first, and derive some orthogonality properties (Theorem 38.1). From these, the claim about minimality of $\|e_n\|_A$ follows as a corollary (Theorem 38.2), and the motivation appears belatedly as we interpret CG as a nonlinear optimization algorithm.

The Conjugate Gradient Iteration

Here is the iteration that Hestenes and Stiefel made famous.

Algorithm 38.1. Conjugate Gradient (CG) Iteration

$x_0 = 0, \;\; r_0 = b, \;\; p_0 = r_0$

for $n = 1, 2, 3, \ldots$

$\alpha_n = (r_{n-1}^T r_{n-1})/(p_{n-1}^T A p_{n-1})$	step length
$x_n = x_{n-1} + \alpha_n p_{n-1}$	approximate solution
$r_n = r_{n-1} - \alpha_n A p_{n-1}$	residual
$\beta_n = (r_n^T r_n)/(r_{n-1}^T r_{n-1})$	improvement this step
$p_n = r_n + \beta_n p_{n-1}$	search direction

Before analyzing the mathematical properties of these formulas, let us examine them operationally. First we note that the CG iteration is extraordinarily simple—programmable in a few lines of MATLAB. Since it deals only with m-vectors, not with individual entries of vectors or matrices, it is simpler, for example, than Gaussian elimination with pivoting. The only complication—which we shall not address—is the choice of a convergence criterion.

At each step, the CG iteration involves several vector manipulations and one matrix-vector product, the computation of Ap_{n-1} (which appears twice in the listing but need be computed only once). If A is dense and unstructured, this matrix-vector product dominates the operation count, which becomes $\sim 2m^2$ flops for each step. If A is sparse or has other exploitable structure, Ap_{n-1} may be computable in as few as $O(m)$ operations, in which case the operation count may be as low as $O(m)$ flops per step.

From the five lines that define the algorithm, the following properties can be deduced. Like all the theorems in this book that do not explicitly mention rounding errors, this one assumes that the computation is performed in exact arithmetic. If there are rounding errors, these properties fail, and it becomes a subtle matter to explain the still very impressive performance of CG.

Theorem 38.1. *Let the CG iteration (Algorithm* 38.1*) be applied to a symmetric positive definite matrix problem $Ax = b$. As long as the iteration has not yet converged (i.e., $r_{n-1} \neq 0$), the algorithm proceeds without divisions by zero, and we have the following identities of subspaces:*

$$\begin{aligned} \mathcal{K}_n &= \langle x_1, x_2, \ldots, x_n \rangle = \langle p_0, p_1, \ldots, p_{n-1} \rangle \\ &= \langle r_0, r_1, \ldots, r_{n-1} \rangle = \langle b, Ab, \ldots, A^{n-1}b \rangle. \end{aligned} \tag{38.3}$$

Moreover, the residuals are orthogonal,

$$r_n^T r_j = 0 \qquad (j < n), \tag{38.4}$$

and the search directions are "A-conjugate,"

$$p_n^T A p_j = 0 \qquad (j < n). \tag{38.5}$$

Proof. The proof is by induction on n; we sketch it informally. From the initial guess $x_0 = 0$ and the formula $x_n = x_{n-1} + \alpha_n p_{n-1}$, it follows by induction that x_n belongs to $\langle p_0, p_1, \ldots, p_{n-1} \rangle$. From $p_n = r_n + \beta_n p_{n-1}$ it follows that this is the same as $\langle r_0, r_1, \ldots, r_{n-1} \rangle$. From $r_n = r_{n-1} - \alpha_n A p_{n-1}$, finally, it follows that this is the same as $\langle b, Ab, \ldots, A^{n-1}b \rangle$. This establishes (38.3).

To prove (38.4), we apply the formula $r_n = r_{n-1} - \alpha_n A p_{n-1}$ and the identity $(Ap_{n-1})^T = p_{n-1}^T A$ to compute

$$r_n^T r_j = r_{n-1}^T r_j - \alpha_n p_{n-1}^T A r_j.$$

If $j < n-1$, both terms on the right are zero by induction. If $j = n-1$, the difference on the right is zero provided $\alpha_n = (r_{n-1}^T r_{n-1})/(p_{n-1}^T A r_{n-1})$. Now this is the same as the line $\alpha_n = (r_{n-1}^T r_{n-1})/(p_{n-1}^T A p_{n-1})$ of Algorithm 38.1, except that $p_{n-1}^T A p_{n-1}$ has been replaced by $p_{n-1}^T A r_{n-1}$. Since p_{n-1} and r_{n-1} differ by $\beta_{n-1} p_{n-2}$, the effect of this replacement is to change the denominator by $\beta_{n-1} p_{n-1}^T A p_{n-2}$, which is zero by the induction hypothesis.

To prove (38.5), we apply the formula $p_n = r_n + \beta_n p_{n-1}$ to compute

$$p_n^T A p_j \;=\; r_n^T A p_j + \beta_n p_{n-1}^T A p_j.$$

If $j < n-1$, both terms on the right are again zero by induction (since (38.4) has now been established for case n). If $j = n-1$, the sum on the right is zero provided $\beta_n = -(r_n^T A p_{n-1})/(p_{n-1}^T A p_{n-1})$, which we can write equivalently in the form $\beta_n = (-\alpha_n r_n^T A p_{n-1})/(\alpha_n p_{n-1}^T A p_{n-1})$. This is the same as the line $\beta_n = (r_n^T r_n)/(r_{n-1}^T r_{n-1})$ of Algorithm 38.1, except that $r_n^T r_n$ has been replaced by $r_n^T(-\alpha_n A p_{n-1})$ and $r_{n-1}^T r_{n-1}$ has been replaced by $p_{n-1}^T(\alpha_n A p_{n-1})$. By the induction hypothesis and lines 3 and 5 of Algorithm 38.1, these replacements can again readily be shown to have zero effect. □

Optimality of CG

In deriving the orthogonality properties (38.4) and (38.5), we have finished the real work. It is now a straightforward matter to confirm that CG minimizes $\|e\|_A$ at each step.

Theorem 38.2. *Let the CG iteration be applied to a symmetric positive definite matrix problem $Ax = b$. If the iteration has not already converged (i.e., $r_{n-1} \neq 0$), then x_n is the unique point in $\mathcal{K}_n$ that minimizes $\|e_n\|_A$. The convergence is monotonic,*

$$\|e_n\|_A \le \|e_{n-1}\|_A, \tag{38.6}$$

and $e_n = 0$ is achieved for some $n \le m$.

Proof. From Theorem 38.1 we know that x_n belongs to $\mathcal{K}_n$. To show that it is the unique point in $\mathcal{K}_n$ that minimizes $\|e\|_A$, consider an arbitrary point $x = x_n - \Delta x \in \mathcal{K}_n$, with error $e = x_* - x = e_n + \Delta x$. We calculate

$$\begin{aligned} \|e\|_A^2 \;&=\; (e_n + \Delta x)^T A (e_n + \Delta x) \\ &=\; e_n^T A e_n + (\Delta x)^T A (\Delta x) + 2 e_n^T A (\Delta x). \end{aligned}$$

The final term in this equation is $2r_n^T(\Delta x)$, an inner product of r_n with a vector in $\mathcal{K}_n$, and by Theorem 38.1, any such inner product is zero. This is the crucial orthogonality property that makes the CG iteration so powerful. It implies that we have

$$\|e\|_A^2 \;=\; e_n^T A e_n + (\Delta x)^T A (\Delta x).$$

Only the second of these terms depends on Δx, and since A is positive definite, that term is ≥ 0, attaining the value 0 if and only if $\Delta x = 0$, i.e., $x_n = x$. Thus $\|e\|_A$ is minimal if and only if $x_n = x$, as claimed.

The remaining statements of the theorem now follow readily. The monotonicity property (38.6) is a consequence of the inclusion $\mathcal{K}_n \subseteq \mathcal{K}_{n+1}$, and since $\mathcal{K}_n$ is a subset of $\mathbb{R}^m$ of dimension n as long as convergence has not yet been achieved, convergence must be achieved in at most m steps. □

The guarantee that the CG iteration converges in at most m steps is void in floating point arithmetic. For arbitrary matrices A on a real computer, no decisive reduction in $\|e_n\|_A$ will necessarily be observed at all when $n = m$. In practice, however, CG is used not for arbitrary matrices but for matrices whose spectra, perhaps thanks to preconditioning, are well-enough behaved that convergence to a desired accuracy is achieved for $n \ll m$ (Lecture 32). The theoretical exact convergence at $n = m$ has no relevance to this use of the CG iteration in scientific computing.

CG as an Optimization Algorithm

We have just shown that the CG iteration has a certain optimality property: it minimizes $\|e_n\|_A$ at step n over all vectors $x \in \mathcal{K}_n$. In fact, as foreshadowed already by the use of such terms as "step length" and "search direction," this iteration can be interpreted as an algorithm of a standard form for minimizing a nonlinear function of $x \in \mathbb{R}^m$. At the heart of the iteration is the formula

$$x_n = x_{n-1} + \alpha_n p_{n-1}.$$

This is a familiar equation in optimization, in which a current approximation x_{n-1} is updated to a new approximation x_n by moving a distance α_n (the step length) in the direction p_{n-1} (the search direction). By a succession of such steps, the CG iteration attempts to find a minimum of a nonlinear function.

Which function? According to Theorem 38.2, the answer would appear to be $\|e\|_A$, or equivalently, $\|e\|_A^2$. However, although $\|e\|_A^2$ is indeed a function of x, it is not one we can evaluate without knowing x_*. It would not be very "standard" to interpret CG as an optimization process applied to a function that cannot be evaluated!

On the other hand, given A and b and $x \in \mathbb{R}^m$, the quantity

$$\varphi(x) = \tfrac{1}{2} x^T A x - x^T b \tag{38.7}$$

can certainly be evaluated. A short computation now reveals

$$\begin{aligned} \|e_n\|_A^2 &= e_n^T A e_n = (x_* - x_n)^T A (x_* - x_n) \\ &= x_n^T A x_n - 2 x_n^T A x_* + x_*^T A x_* \\ &= x_n^T A x_n - 2 x_n^T b + x_*^T b = 2\varphi(x_n) + \text{constant}. \end{aligned}$$

Thus $\varphi(x)$ is the same as $\|e\|_A^2$ except for a factor of 2 and the (unknown) constant $x_*^T b$. Like $\|e\|_A^2$, it must achieve its minimum (namely, $-x_*^T b/2$) uniquely at $x = x_*$.

The CG iteration can be interpreted as an iterative process for minimizing the quadratic function $\varphi(x)$ of $x \in \mathbb{R}^m$. At each step, an iterate $x_n = x_{n-1} + \alpha_n p_{n-1}$ is computed that minimizes $\varphi(x)$ over all x in the one-dimensional space $x_{n-1} + \langle p_{n-1} \rangle$. (It is readily confirmed that the formula $\alpha_n = (r_{n-1}^T r_{n-1})/(p_{n-1}^T A p_{n-1})$ ensures that α_n is optimal in this sense among all step lengths α.) What makes the CG iteration remarkable is the choice of the search direction p_{n-1}, which has the special property that minimizing $\varphi(x)$ over $x_{n-1} + \langle p_{n-1} \rangle$ actually minimizes it over all of $\mathcal{K}_n$.

There is a close analogy between the CG iteration for solving $Ax = b$ and the Lanczos iteration for finding eigenvalues. The eigenvalues of A, as discussed in Lecture 27, are the stationary values for $x \in \mathbb{R}^m$ of the Rayleigh quotient, $r(x) = (x^T A x)/(x^T x)$. As pointed out in Exercise 36.1, the eigenvalue estimates (Ritz values) associated with step n of the Lanczos iteration are the stationary values of the same function $r(x)$ if x is restricted to the Krylov subspace $\mathcal{K}_n$. This is a perfect parallel of what we have shown in the last two pages, that the solution x_* of $Ax = b$ is the minimal point in $\mathbb{R}^m$ of the scalar function $\varphi(x)$, and the CG iterate x_n is the minimal point of the same function $\varphi(x)$ if x is restricted to $\mathcal{K}_n$.

CG and Polynomial Approximation

A theme of the last four lectures has been the connection between Krylov subspace iterations and polynomials of matrices. The Arnoldi and Lanczos iterations solve the Arnoldi/Lanczos approximation problem (34.3), and the GMRES iteration solves the GMRES approximation problem (35.10). For CG, the appropriate approximation problem involves the A-norm of the error.

CG Approximation Problem. Find $p_n \in P_n$ such that

$$\|p_n(A)e_0\|_A = \text{minimum}. \tag{38.8}$$

Here e_0 denotes the initial error, $e_0 = x_* - x_0 = x_*$, and P_n is again defined as in (35.7), the set of polynomials p of degree $\le n$ with $p(0) = 1$. From Theorem 38.2 we may derive the following convergence theorem.

Theorem 38.3. *If the CG iteration has not already converged before step n (i.e., $r_{n-1} \neq 0$), then* (38.8) *has a unique solution $p_n \in P_n$, and the iterate x_n has error $e_n = p_n(A)e_0$ for this same polynomial p_n. Consequently we have*

$$\frac{\|e_n\|_A}{\|e_0\|_A} = \inf_{p \in P_n} \frac{\|p(A)e_0\|_A}{\|e_0\|_A} \le \inf_{p \in P_n} \max_{\lambda \in \Lambda(A)} |p(\lambda)|, \tag{38.9}$$

where $\Lambda(A)$ denotes the spectrum of A.

Proof. From Theorem 38.1 it follows that $e_n = p(A)e_0$ for some $p \in P_n$. The equality in (38.9) is a consequence of this and Theorem 38.2. As for the inequality in (38.9), if $e_0 = \sum_{j=1}^m a_j v_j$ is an expansion of e_0 in orthonormal eigenvectors of A, then we have $p(A)e_0 = \sum_{j=1}^m a_j p(\lambda_j) v_j$ and thus

$$\|e_0\|_A^2 = \sum_{j=1}^m a_j^2 \lambda_j, \qquad \|p(A)e_0\|_A^2 = \sum_{j=1}^m a_j^2 \lambda_j (p(\lambda_j))^2.$$

These identities imply $\|p(A)e_0\|_A^2 / \|e_0\|_A^2 \le \max_{\lambda \in \Lambda(A)} |p(\lambda)|^2$, which implies the inequality in question. □

Rate of Convergence

Theorem 38.3 establishes that the rate of convergence of the CG iteration is determined by the location of the spectrum of A. A good spectrum is one on which polynomials $p_n \in P_n$ can be very small, with size decreasing rapidly with n. Roughly speaking, this may happen for either or both of two reasons: the eigenvalues may be grouped in small clusters, or they may lie well separated in a relative sense from the origin. The two best-known corollaries of Theorem 38.3 address these two ideas in their extreme forms.

First, we suppose that the eigenvalues are perfectly clustered but assume nothing about the locations of these clusters.

Theorem 38.4. *If A has only n distinct eigenvalues, then the CG iteration converges in at most n steps.*

Proof. This is a corollary of (38.9), since a polynomial $p(x) = \prod_{j=1}^n (1 - x/\lambda_j) \in P_n$ exists that is zero at any specified set of n points $\{\lambda_j\}$. □

At the other extreme, suppose we know nothing about any clustering of the eigenvalues but only that their distances from the origin vary by at most a factor $\kappa \ge 1$. In other words, suppose we know only the 2-norm condition number $\kappa = \lambda_{\max}/\lambda_{\min}$, where $\lambda_{\max}$ and $\lambda_{\min}$ are the extreme eigenvalues of A.

Theorem 38.5. *Let the CG iteration be applied to a symmetric positive definite matrix problem $Ax = b$, where A has 2-norm condition number κ. Then the A-norms of the errors satisfy*

$$\frac{\|e_n\|_A}{\|e_0\|_A} \le 2 \Big/ \left[\left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1} \right)^n + \left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1} \right)^{-n} \right] \le 2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^n. \quad (38.10)$$

Proof. By Theorem 38.3, it is enough to find a polynomial $p \in P_n$ whose maximum value for $\lambda \in [\lambda_{\min}, \lambda_{\max}]$ is the middle expression of (38.10). The polynomial we choose is the scaled and shifted Chebyshev polynomial $p(x) = T_n(\gamma - 2x/(\lambda_{\max} - \lambda_{\min}))/T_n(\gamma)$, where T_n is the usual Chebyshev polynomial of degree n and γ takes the special value $\gamma = (\lambda_{\max} + \lambda_{\min})/(\lambda_{\max} - \lambda_{\min}) = (\kappa + 1)/(\kappa - 1)$. For $x \in [\lambda_{\min}, \lambda_{\max}]$, the argument of T_n in the numerator of $p(x)$ lies in $[-1, 1]$, which means the magnitude of that numerator is ≤ 1. Therefore, to prove the theorem, it will suffice to show

$$T_n(\gamma) \;=\; T_n\left(\frac{\kappa+1}{\kappa-1}\right) \;=\; \frac{1}{2}\left[\left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}\right)^n + \left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}\right)^{-n}\right]. \qquad (38.11)$$

We can do this by making the change of variables $x = \frac{1}{2}(z + z^{-1})$, $T_n(x) = \frac{1}{2}(z^n + z^{-n})$, standard in the study of Chebyshev polynomials. If $(\kappa+1)/(\kappa-1) = \frac{1}{2}(z + z^{-1})$, that is, $\frac{1}{2}z^2 - (\kappa+1)/(\kappa-1)z + \frac{1}{2} = 0$, then we have a quadratic equation with solution

$$\begin{aligned} z \;&=\; \left(\frac{\kappa+1}{\kappa-1}\right) + \sqrt{\left(\frac{\kappa+1}{\kappa-1}\right)^2 - 1} \;=\; \frac{\kappa+1+\sqrt{(\kappa+1)^2-(\kappa-1)^2}}{\kappa-1} \\ &=\; \frac{\kappa+1+\sqrt{4\kappa}}{\kappa-1} \;=\; \frac{(\sqrt{\kappa}+1)^2}{(\sqrt{\kappa}+1)(\sqrt{\kappa}-1)} \;=\; \frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}. \end{aligned}$$

Thus $T_n(\gamma) = \frac{1}{2}(z^n + z^{-n})$ for this value of z, which is (38.11), as claimed. □

Theorem 38.5 is the most famous result about convergence of the CG iteration. Since

$$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \;\sim\; 1 - \frac{2}{\sqrt{\kappa}}$$

as $\kappa \to \infty$, it implies that if κ is large but not too large, convergence to a specified tolerance can be expected in $O(\sqrt{\kappa})$ iterations. One must remember that this is only an upper bound. Convergence may be faster for special right-hand sides (not so common) or if the spectrum is clustered (more common).

Example

For an example of the convergence of CG, consider a 500×500 sparse matrix A constructed as follows. First we put 1 at each diagonal position and a random number from the uniform distribution on $[-1, 1]$ at each off-diagonal position (maintaining the symmetry $A = A^T$). Then we replace each off-diagonal entry with $|a_{ij}| > \tau$ by zero, where τ is a parameter. For τ close to zero, the result is a well-conditioned positive definite matrix whose density of nonzero entries is approximately τ. As τ increases, both the condition number and the sparsity deteriorate.

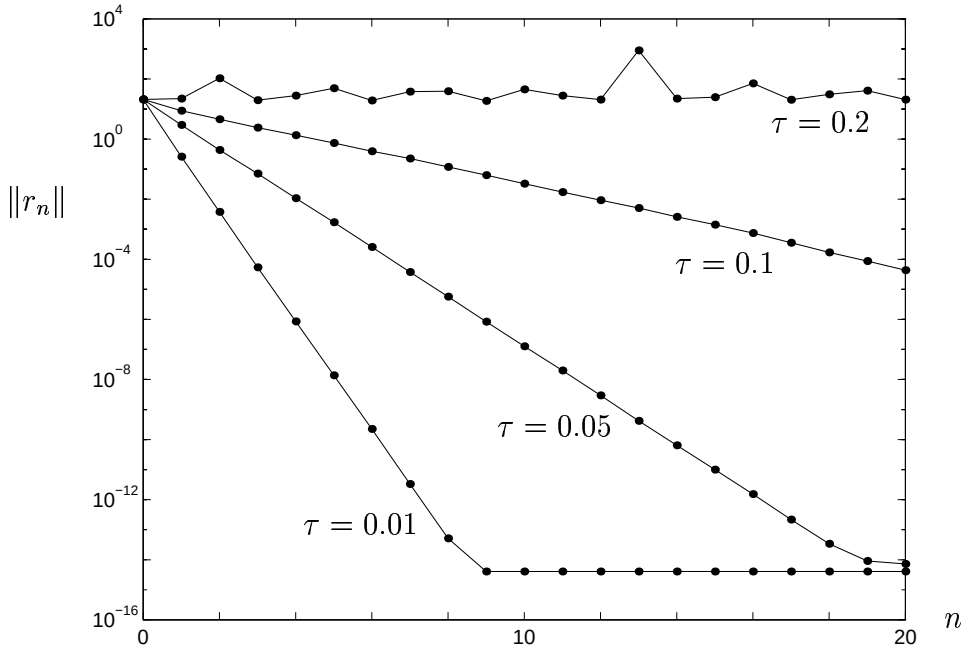


Figure 38.1. *CG convergence curves for the* 500×500 *sparse matrices* A *described in the text. For* $\tau = 0.01$*, the system is solved about* 700 *times faster by CG than by Cholesky factorization. For* $\tau = 0.2$*, the matrix is not positive definite and there is no convergence.*

Figure 38.1 shows convergence curves corresponding to 20 steps of the CG iteration for matrices of this kind with $\tau = 0.01, 0.05, 0.1, 0.2$. (The right-hand side b was taken to be a random vector.) For $\tau = 0.01$, A has 3092 nonzero entries and condition number $\kappa \approx 1.06$. Convergence to machine precision takes place in 9 steps, about 6×10^4 flops. For $\tau = 0.05$, there are 13,062 nonzeros with $\kappa \approx 1.83$, and convergence takes 19 steps, about 5×10^5 flops. For $\tau = 0.1$ we have 25,526 nonzeros and $\kappa \approx 10.3$, with only 5 digits of convergence after 20 steps and 10^6 flops. For $\tau = 0.2$, with 50,834 nonzeros, there is no convergence at all. The lowest eigenvalue is now negative, so A is no longer positive definite and the use of the CG iteration is inappropriate. (In fact, the CG iteration often succeeds with indefinite matrices, but in this case the matrix is not only indefinite but ill-conditioned.)

Note how closely the $\tau = 0.01$ curve of Figure 38.1 matches the schematic ideal depicted in Figure 32.1! For this example, the operation count of 6×10^4 flops beats Cholesky factorization (23.4) by a factor of about 700. Unfortunately, not every matrix arising in practice has such a well-behaved spectrum, even after the best efforts to find a good preconditioner.

Exercises

38.1. Based on the condition numbers κ reported in the text, determine the rate of convergence predicted by Theorem 38.5 for the matrices A of Figure 38.1 with $\tau = 0.01, 0.05, 0.1$. Draw lines on a copy of Figure 38.1 indicating how closely these predictions match the actual convergence rates.

38.2. Suppose A is a real symmetric 805×805 matrix with eigenvalues $1.00, 1.01, 1.02, \ldots, 8.98, 8.99, 9.00$ and also $10, 12, 16, 24$. How many steps of the conjugate gradient iteration must you take to be sure of reducing the initial error $\|e_0\|_A$ by a factor of 10^6?

38.3. The conjugate gradient is applied to a symmetric positive definite matrix A with the result $\|e_0\|_A = 1$, $\|e_{10}\|_A = 2 \times 2^{-10}$. Based solely on this data,

(a) What bound can you give on $\kappa(A)$?

(b) What bound can you give on $\|e_{20}\|_A$?

38.4. Suppose A is a dense symmetric positive definite 1000×1000 matrix with $\kappa(A) = 100$. Estimate roughly how many flops are required to solve $Ax = b$ to ten-digit accuracy by (a) Cholesky factorization, (b) Richardson iteration with the optimal parameter α (Exercise 35.3), and (c) CG.

38.5. We have described CG as an iterative minimization of the function $\varphi(x)$ of (38.7). Another way to minimize the same function—far slower, in general—is by the method of *steepest descent*.

(a) Derive the formula $\nabla\varphi(x) = -r$ for the gradient of $\varphi(x)$. Thus the steepest descent iteration corresponds to the choice $p_n = r_n$ instead of $p_n = r_n + \beta_n p_{n-1}$ in Algorithm 38.1.

(b) Determine the formula for the optimal step length α_n of the steepest descent iteration.

(c) Write down the full steepest descent iteration. There are three operations inside the main loop.

38.6. Let A be the 100×100 tridiagonal symmetric matrix with $1, 2, \ldots, 100$ on the diagonal and 1 on the sub- and superdiagonals, and set $b = (1, 1, \ldots, 1)^T$. Write a program that takes 100 steps of the CG and also the steepest descent iteration to approximately solve $Ax = b$. Produce a plot with four curves on it: the computed residual norms $\|r_n\|_2$ for CG, the actual residual norms $\|b - Ax_n\|_2$ for CG, the residual norms $\|r_n\|_2$ for steepest descent, and the estimate $2(\sqrt{\kappa} - 1)^n/(\sqrt{\kappa} + 1)^n$ of Theorem 38.5. Comment on your results.

Lecture 39. Biorthogonalization Methods

Not all Krylov subspace iterations for nonsymmetric systems involve recurrences of growing length and growing cost. Methods based on three-term recurrences have also been devised, and they are the most powerful nonsymmetric iterations available today. The price to be paid, at least for some of the iterations in this category, is that one must work with two Krylov subspaces rather than one, generated by multiplications by A^* as well as A.

Where We Stand

On p. 245 we presented a table of Krylov subspace matrix iterations:

	$Ax = b$	$Ax = \lambda x$
$A = A^*$	CG	Lanczos
$A \neq A^*$	GMRES CGN BCG et al.	Arnoldi

Our discussions of three of these boxes are now complete, and as for the fourth, lower-left position, we have already discussed GMRES. In this lecture we turn to the final two lines of the table. We spend just a moment on CGN,

a simple and easily analyzed algorithm, and then move to our main subject, the biorthogonalization methods represented by the entry "BCG et al."

CGN = CG Applied to the Normal Equations

Let $A \in \mathbb{C}^{m\times m}$ be nonsingular but not necessarily hermitian, so that $Ax = b$, for any $b \in \mathbb{C}^m$, is a nonsingular square system of equations. One of the simplest methods for solving such a system is to apply the CG iteration to the normal equations (11.9),

$$A^*Ax = A^*b. \tag{39.1}$$

(The matrix A^*A is not formed explicitly, which would require m^3 flops. Instead, each matrix-vector product A^*Av is evaluated in two steps as $A^*(Av)$.) Since A is nonsingular, A^*A is hermitian positive definite, or symmetric positive definite if A is real. Thus the theorems of the last lecture apply, and rapid convergence can be expected if the eigenvalues of A^*A are favorably distributed. This method goes by the name of CGN (also CGNR), which roughly stands for "CG applied to the normal equations."

Since we have already analyzed the behavior of CG, nothing new is needed to understand the behavior of CGN. If the initial guess is $x_0 = 0$, as in Algorithm 38.1, then from Theorem 38.1 we see that the later iterates belong to a Krylov subspace generated by A^*A:

$$x_n \in \langle A^*b,\ (A^*A)A^*b,\ \ldots,\ (A^*A)^{n-1}A^*b\rangle. \tag{39.2}$$

From Theorem 38.2 we know that the A^*A-norm of the error is minimized over this space at each step, and since $\|e_n\|^2_{A^*A} = e_n^*A^*Ae_n = \|Ae_n\|_2^2 = \|r_n\|^2$, this is another way of saying that the 2-norm of the residual $r_n = b - Ax_n$ is minimized:

$$\|r_n\|_2 \ = \ \text{minimum}. \tag{39.3}$$

Thus CGN, like GMRES, is a minimal residual method, but since the Krylov subspaces (33.5) and (39.2) are different, these two methods are by no means equivalent.

According to Theorem 38.3, the convergence of CGN is controlled by the eigenvalues of A^*A. These numbers are equal to the squares of the singular values of A. Thus the convergence of CGN is determined by the singular values of A, and in principle has nothing to do with the eigenvalues of A. The fact that squares are involved is unfortunate, however. If A has condition number κ, then A^*A has condition number κ^2, and the analogue of (38.10) for CGN becomes

$$\frac{\|r_n\|_2}{\|r_0\|_2} \ \le \ 2\left(\frac{\kappa-1}{\kappa+1}\right)^n. \tag{39.4}$$

For large κ, this is far worse than (38.10); it implies that $O(\kappa)$ iterations are required for convergence to a fixed accuracy, not $O(\sqrt{\kappa}\,)$.

This "squaring of the condition number" has given the CGN iteration a poor reputation, which, on balance, may be deserved. Nevertheless, for some problems CGN vastly outperforms alternative methods, since their convergence depends on eigenvalues rather than singular values. All one needs is a matrix whose singular values are well behaved but whose eigenvalues are not, such as a well-conditioned matrix whose spectrum surrounds the origin in the complex plane. An extreme example is provided by the $m \times m$ circulant matrix of the form

$$A = \begin{bmatrix} 0 & 1 & & & \\ & 0 & 1 & & \\ & & 0 & 1 & \\ & & & 0 & 1 \\ 1 & & & & 0 \end{bmatrix}. \tag{39.5}$$

The singular values of this matrix are all equal to 1, but the eigenvalues are the mth roots of unity. GMRES requires m steps for convergence for a general right-hand side b, while CGN converges in one step. (See Exercise 39.1.)

Another virtue of the CGN iteration is that since it is based on the normal equations, it applies without modification to least squares problems (cf. Algorithm 11.1), where A is no longer square. Alternatively, some iterative methods for least squares problems are based on the block system (19.4) of Exercise 19.1.

Tridiagonal Biorthogonalization

The Lanczos iteration, as we saw in Lecture 36, is a process of *tridiagonal orthogonalization*. If carried a full m steps (in exact arithmetic), it would produce a unitary reduction (36.5) of a hermitian matrix to tridiagonal form: $A = QTQ^*$.

If A is not hermitian, such a reduction is not possible in general: we must give up either the unitary transformations or the final tridiagonal form. The Arnoldi iteration, a process of *Hessenberg orthogonalization*, does the latter. If carried a full m steps, it would produce a unitary reduction (33.12) of an arbitrary square matrix to Hessenberg form: $A = QHQ^*$.

Biorthogonalization methods are based on the opposite choice. If we insist on a tridiagonal result but give up the use of unitary transformations, we have a process of *tridiagonal biorthogonalization*: $A = VTV^{-1}$, where V is nonsingular but generally not unitary (Figure 39.1). Taking the adjoint gives the equivalent equation $A^* = V^{-*}T^*(V^{-*})^{-1}$. (Recall from p. 12 that $V^{-*} = (V^*)^{-1} = (V^{-1})^*$.) The term "biorthogonal" refers to the fact that although the columns of V are not orthogonal to each other, they are orthogonal to the columns of V^{-*}, as follows trivially from the identity $(V^{-*})^*V = V^{-1}V = I$.

To begin to make this idea into an iterative algorithm, we must see what is involved for $n < m$. Let V be a nonsingular matrix such that $A = VTV^{-1}$

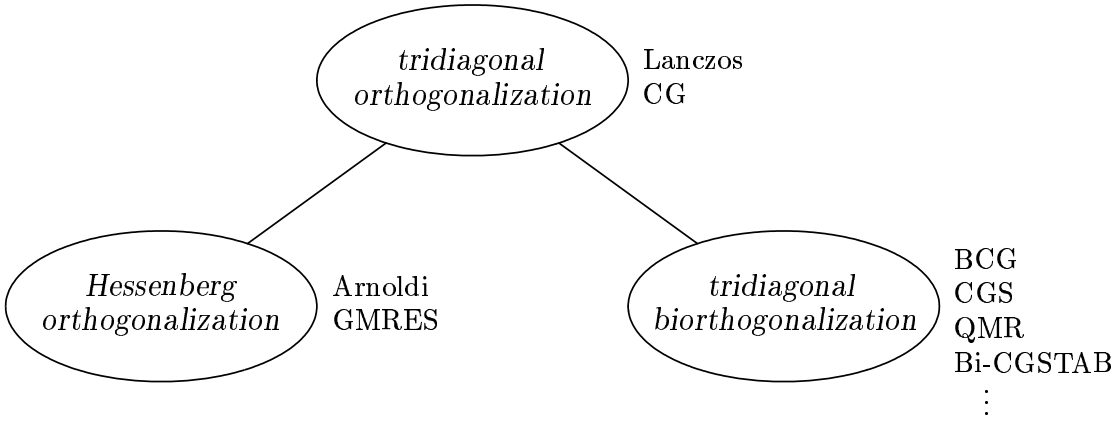


Figure 39.1. *Classification of Krylov subspace iterations. If the matrix is hermitian (top row), then it can be orthogonalized by a three-term recurrence relation—a tridiagonal matrix. If it is not hermitian, one must give up either the tridiagonal structure or the orthogonality.*

with T tridiagonal, and define $W = V^{-*}$. Let v_j and w_j denote the jth columns of V and W, respectively. These vectors are *biorthogonal* in the sense that

$$w_i^* v_j = \delta_{ij}, \tag{39.6}$$

where δ_{ij} is the Kronecker delta function (p. 14). For each n with $1 \le n \le m$, following (33.1), define the $m \times n$ matrices

$$V_n = \left[\begin{array}{c|c|c} & & \\ v_1 & \cdots & v_n \\ & & \end{array}\right], \qquad W_n = \left[\begin{array}{c|c|c} & & \\ w_1 & \cdots & w_n \\ & & \end{array}\right]. \tag{39.7}$$

In matrix form, the biorthogonality condition can be written

$$W_n^* V_n \;=\; V_n^* W_n \;=\; I_n,$$

where I_n is the identity of dimension n.

We can now write down the key formulas that are the basis of biorthogonalization methods. For the Lanczos iteration, we had (36.5) and (36.6),

$$AQ_n = Q_{n+1}\tilde{T}_n, \qquad T_n = Q_n^* A Q_n.$$

For the Arnoldi iteration, we had (33.12) and (33.13),

$$AQ_n = Q_{n+1}\tilde{H}_n, \qquad H_n = Q_n^* A Q_n.$$

These are the corresponding formulas for biorthogonalization methods:

$$AV_n = V_{n+1}\tilde{T}_n, \tag{39.8}$$

$$A^*W_n = W_{n+1}\tilde{S}_n, \tag{39.9}$$

$$T_n = S_n^* = W_n^* A V_n. \tag{39.10}$$

Here V_n and W_n have dimensions $m \times n$, $\tilde{T}_{n+1}$ and $\tilde{S}_{n+1}$ are (nonhermitian) tridiagonal matrices with dimensions $(n+1) \times n$, and $T_n = S_n^*$ is the $n \times n$ matrix obtained by deleting the last row of $\tilde{T}_{n+1}$ or the last column of $\tilde{S}_{n+1}^*$.

In analogy to the developments of p. 252, (39.8) can be displayed as

$$\begin{bmatrix} & & \\ & A & \\ & & \end{bmatrix} \left[\begin{array}{c|c|c} & & \\ v_1 & \cdots & v_n \\ & & \end{array}\right] = \left[\begin{array}{c|c|c} & & \\ v_1 & \cdots & v_{n+1} \\ & & \end{array}\right] \begin{bmatrix} \alpha_1 & \gamma_1 & & & \\ \beta_1 & \alpha_2 & \gamma_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \gamma_{n-1} \\ & & & \beta_{n-1} & \alpha_n \\ & & & & \beta_n \end{bmatrix},$$

which corresponds to the three-term recurrence relation

$$Av_n = \gamma_{n-1}v_{n-1} + \alpha_n v_n + \beta_n v_{n+1}. \tag{39.11}$$

Similarly, (39.9) takes the form

$$\begin{bmatrix} & & \\ & A^* & \\ & & \end{bmatrix} \left[\begin{array}{c|c|c} & & \\ w_1 & \cdots & w_n \\ & & \end{array}\right] = \left[\begin{array}{c|c|c} & & \\ w_1 & \cdots & w_{n+1} \\ & & \end{array}\right] \begin{bmatrix} \overline{\alpha}_1 & \overline{\beta}_1 & & & \\ \overline{\gamma}_1 & \overline{\alpha}_2 & \overline{\beta}_2 & & \\ & \overline{\gamma}_2 & \overline{\alpha}_3 & \ddots & \\ & & \ddots & \ddots & \overline{\beta}_{n-1} \\ & & & \overline{\gamma}_{n-1} & \overline{\alpha}_n \\ & & & & \overline{\gamma}_n \end{bmatrix},$$

corresponding to

$$A^*w_n = \overline{\beta}_{n-1}w_{n-1} + \overline{\alpha}_n w_n + \overline{\gamma}_n w_{n+1}. \tag{39.12}$$

(We have not seen these bars for complex conjugation before, because in the last three lectures, we assumed that A was real.)

As usual with Krylov subspace iterations, these equations suggest an algorithm. Begin with vectors v_1 and w_1 that are arbitrary except for satisfying $v_1^*w_1 = 1$, and set $\beta_0 = \gamma_0 = 0$ and $v_0 = w_0 = 0$. Now, for each $n = 1, 2, \ldots$, set $\alpha_n = w_n^*Av_n$, as follows from (39.6) and (39.11) or (39.12). The vectors v_{n+1} and w_{n+1} are then determined by (39.11) and (39.12) up to scalar

factors. These factors may be chosen arbitrarily, subject to the normalization $w_{n+1}^* v_{n+1} = 1$, whereupon β_{n+1} and γ_{n+1} are determined by (39.11) and (39.12).

The vectors generated by the procedure just described lie in the Krylov subspaces

$$v_n \in \langle v_1,\ Av_1, \ldots, A^{n-1}v_1 \rangle, \qquad w_n \in \langle w_1,\ A^*w_1, \ldots, (A^*)^{n-1}w_1 \rangle. \tag{39.13}$$

For a generic matrix, in exact arithmetic, the procedure will run to completion after m steps, but for certain special matrices there may also be a breakdown of the process before this point. If $v_n = 0$ or $w_n = 0$ at some step, an invariant subspace of A or A^* has been found: the tridiagonal matrix T is reducible (cf. Exercise 33.2). Alternatively, it may also happen that $v_n \neq 0$ and $w_n \neq 0$ but $w_n^* v_n = 0$. The possibility of this more serious kind of breakdown is present in most biorthogonalization methods. As in other areas of numerical analysis, the fact that exact breakdown is possible for certain problems implies that near-breakdown may occur for many other problems, with potentially adverse consequences in floating point arithmetic. Some methods for coping with these phenomena are mentioned at the end of this lecture.

BCG = Biconjugate Gradients

One way to use the biorthogonalization process just described is to compute eigenvalues: as $n \to \infty$, some eigenvalues of T_n may converge rapidly to some eigenvalues of A. Another application, which we shall now briefly discuss, is the solution of nonsingular systems of equations $Ax = b$. The classic algorithm of this type is known as *biconjugate gradients* or *BCG*.

The principle of BCG is as follows. We take $v_1 = b$, so that the first Krylov subspace in (39.13) becomes $\mathcal{K}_n = \langle b, Ab, \ldots, A^{n-1}b \rangle$. Recall that the principle of GMRES is to pick $x_n \in \mathcal{K}_n$ so that the orthogonality condition

$$\text{GMRES:} \qquad r_n \ \perp\ \langle Ab, A^2b, \ldots, A^nb \rangle \ = \ A\mathcal{K}_n \tag{39.14}$$

is satisfied, where $r_n = b - Ax_n$ is the residual corresponding to x_n (Figure 35.1). This choice has the effect of minimizing $\|r_n\|$, the 2-norm of the residual. The principle of the BCG algorithm is to pick x_n in the same subspace, $x_n \in \mathcal{K}_n$, but to enforce the orthogonality condition

$$\text{BCG:} \qquad r_n \ \perp\ \langle w_1, A^*w_1, \ldots, (A^*)^{n-1}w_1 \rangle. \tag{39.15}$$

Here $w_1 \in \mathbb{C}^m$ is an arbitrary vector satisfying $w_1^* v_1 = 1$; in applications one sometimes takes $w_1 = v_1/\|v_1\|_2$. Unlike (39.14), this choice does not minimize $\|r_n\|_2$, and it is not optimal from the point of view of minimizing the number of iterations. Its advantage is that it can be implemented with three-term recurrences rather than the $(n+1)$-term recurrences of GMRES.

Without giving details of the derivation, we now record the BCG algorithm in its standard form. What follows should be compared with Algorithm 38.1, the CG algorithm. The two are the same except that the sequence of search directions $\{p_n\}$ of CG has become two sequences $\{p_n\}$ and $\{q_n\}$, and the sequence of residuals $\{r_n\}$ of CG has become two sequences $\{r_n\}$ and $\{s_n\}$.

Algorithm 39.1. Biconjugate Gradient (BCG) Iteration

$x_0 = 0, \;\; p_0 = r_0 = b, \;\; q_0 = s_0 =$ arbitrary

for $n = 1, 2, 3, \ldots$

$\alpha_n = (s_{n-1}^* r_{n-1})/(q_{n-1}^* A p_{n-1})$

$x_n = x_{n-1} + \alpha_n p_{n-1}$

$r_n = r_{n-1} - \alpha_n A p_{n-1}$

$s_n = s_{n-1} - \overline{\alpha}_n A^* q_{n-1}$

$\beta_n = (s_n^* r_n)/(s_{n-1}^* r_{n-1})$

$p_n = r_n + \beta_n p_{n-1}$

$q_n = s_n + \overline{\beta}_n q_{n-1}$

As in Theorem 38.1, it is readily shown that $s_n^* r_j = 0$ and $q_n^* A p_j = 0$ for $j < n$.

Example

In Figure 38.1 we illustrated the convergence of the CG iteration for a 500×500 sparse symmetric positive definite matrix dependent on a parameter τ. To illustrate the convergence of BCG, consider the same matrix with one change: the signs of all the entries are randomized. This makes the matrix no longer hermitian, and it changes the dominant entries on the diagonal to 1 and -1 at random, rather than all 1, so that the eigenvalues are clustered around 1 and -1 instead of just 1.

Figure 39.2 shows the convergence of GMRES and BCG for such a matrix with $\tau = 0.01$. Considering first the GMRES curve, we note that the convergence is half as fast as in Figure 38.1, with essentially no progress at each odd-numbered step, but steady progress at each even step. This odd–even effect is a result of the approximate ± 1 symmetry of the matrix: a polynomial $p(z)$ of degree $2k+1$ with $p(0) = 1$ can be no smaller at 1 and -1 than a corresponding polynomial of degree $2k$. Turning now to the BCG curve, we see that the convergence is comparable in an overall sense, but it is no longer monotonic, showing spikes of magnitude as great as about 10^2 at each odd-numbered step. The accuracy attained at the end has also suffered by more than a digit. All of these features are typical of BCG computations.

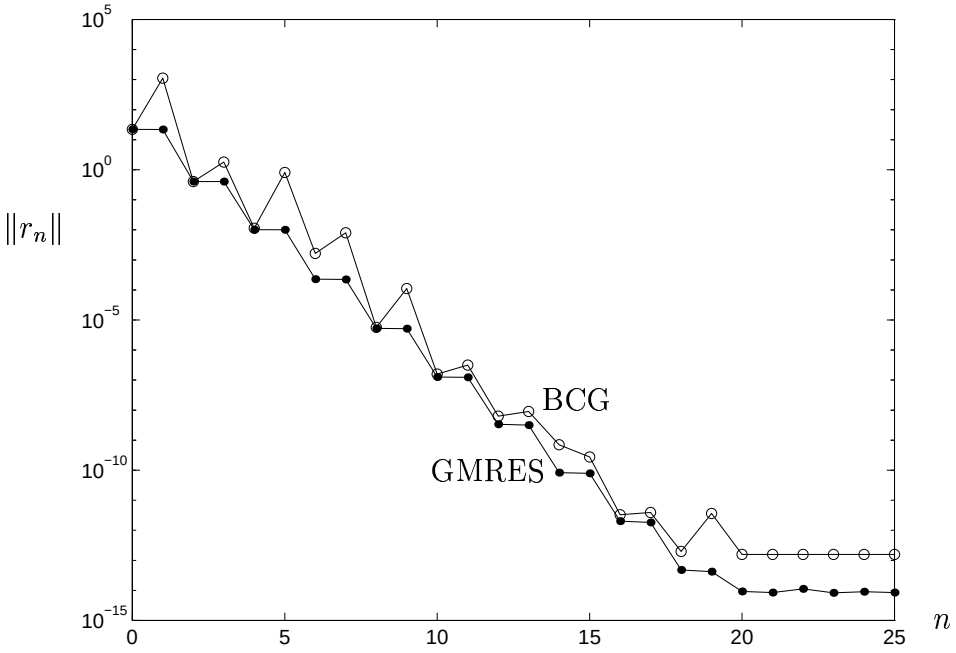


Figure 39.2. *Comparison of GMRES and BCG for the* 500×500 *matrix labeled* $\tau = 0.01$ *in Figure* 38.1*, but with the signs of the entries randomized.*

The horizontal axis in Figure 39.2 is the step number n, which is not the same as the computational cost. At each step, GMRES requires one matrix-vector multiplication involving A, whereas BCG requires multiplications involving both A and A^*. For problems where matrix-vector multiplications dominate the work and enough storage is available, GMRES may consequently be twice as fast as BCG or faster. Here, however, the matrix is sparse enough that the work associated with handling long recurrences is significant, and in fact, the BCG calculation of Figure 39.2 was faster than the GMRES calculation by better than a factor of 2.

QMR and Other Variants

BCG has one great advantage over GMRES: it involves three-term recurrences, enabling the work per step and the storage requirements to remain under control even when many steps are needed. On the other hand, it has two disadvantages. One is that in comparison to the monotonic and often rapid convergence of GMRES as a function of step number, its convergence is slower and often erratic, sometimes far more erratic than in Figure 39.2. Irregular convergence is unattractive, and it may have the consequence of reducing the ultimately attainable accuracy because of rounding errors (Exercise 39.4). In the extreme, it becomes the phenomenon of breakdown of the iteration, where an inner product becomes zero and no further progress is possible, even though

the system of equations may be well-conditioned.

The other problem with BCG is that it requires multiplication by A^* as well as A. Depending on how these products are implemented both mathematically and in terms of computer architecture, this may be anything from a minor additional burden to effectively impossible.

In response to these two problems, beginning in the 1980s, more than a dozen variants of BCG have been proposed. Here are some of the best known of these; references are given in the Notes.

Look-ahead Lanczos (Parlett, Taylor, and Liu, 1985)
CGS = conjugate gradients squared (Sonneveld, 1989)
QMR = quasi-minimal residuals (Freund and Nachtigal, 1991)
Bi-CGSTAB = stabilized BCG (van der Vorst, 1992)
TFQMR = transpose-free QMR (Freund, 1993)

We shall say a few words about these methods but give no details.

The look-ahead Lanczos algorithm is based on the fact that when a breakdown is about to take place, it can be avoided by taking two or more steps of the iteration at once rather than a single step. The original idea of Parlett et al. has been developed extensively by later authors and is incorporated, for example, in the version of the QMR algorithm recommended by its authors. The phenomenon of breakdown can be shown to be equivalent to the phenomenon of square blocks of identical entries in the table of Padé approximants to a function, and the look-ahead idea amounts to a method of stepping across such blocks in one step. In practice, of course, one does not just test for exact breakdowns; a notion of near-breakdown defined by appropriate tolerances is involved.

The CGS algorithm is based on the discovery that if two steps of the BCG are combined into one in a different manner, so that the algorithm is "squared," then multiplication by A^* can be avoided. The result is a "transpose-free" method that sometimes converges up to twice as quickly as BCG, though the convergence is also twice as erratic.

The QMR algorithm is based on the observation that although three-term recurrences cannot be used to minimize $\|r_n\|$, they can be used to minimize a different, data-dependent norm that in practice is usually not so far from $\|r_n\|$. This may have a pronounced effect on the smoothness of convergence, significantly reducing the impact of rounding errors. The Bi-CGSTAB algorithm is another method that also significantly smooths the convergence rate of BCG, and TFQMR is a variant of QMR that combines its smooth convergence with the avoidance of the need for A^*.

Most recently, efforts have been directed at combining these three virtues of smoothed convergence curves, look-ahead to avoid breakdowns, and transpose-free operation. So far, all three have not yet been combined fully satisfactorily in a single algorithm, but this research area is young.

Exercises

39.1. Consider a problem $Ax = b$ for the matrix (39.5) of dimension m.

(a) Show that the singular values are all 1 and that this implies that CGN converges in one step.

(b) Show that the eigenvalues are the mth roots of unity and that this implies that GMRES requires m steps to converge for general b.

(c) This matrix A has so much structure that one does not need to consider eigenvalues or singular values to understand its convergence behavior. In particular, explain by an elementary argument why GMRES takes m steps to converge for the right-hand side $b = (1, 0, 0, \ldots, 0)^T$.

39.2. As a converse to Exercise 39.1, devise an example of a matrix of arbitrary dimension m with almost the opposite property: GMRES converges in two steps, but CGN requires m steps.

39.3. (a) If A is hermitian and s_0 is chosen appropriately, Algorithm 39.1 reduces to Algorithm 38.1. Confirm this statement and determine the appropriate s_0.

(b) Suppose A is a complex matrix that is symmetric but not hermitian. Show that with a different choice of s_0, Algorithm 39.1 again reduces to an iteration involving just one three-term recurrence.

39.4. Figure 39.2 illustrated that if the convergence curve for a biorthogonalization method has spikes in it, this may affect the attainable accuracy in floating point arithmetic. Without trying to be rigorous, explain why this is so, and comment on the analogy with growth factors in Gaussian elimination (Lecture 22).

39.5. Which of CG, GMRES, CGN, or BCG would you expect to be most effective for the following $m \times m$ problems $Ax = b$, and why?

(a) A dense nonhermitian matrix with $m = 10^4$, all but three of whose eigenvalues are approximately equal to -1.

(b) The same, but with all but three of the eigenvalues scattered about the region $-10 \le \text{Real}(\lambda) \le 10$, $-1 \le \text{Imag}(\lambda) \le 1$.

(c) A sparse nonhermitian matrix with $m = 10^6$ but only 10^7 nonzero entries, with eigenvalues as in (a).

(d) A sparse hermitian matrix with $m = 10^5$ whose eigenvalues are scattered through the interval $[1, 100]$.

(e) The same, except for outlying eigenvalues at 0.01 and 10,000.

(f) The same, but with additional outliers at -1, -10, and -100.

(g) A sparse, normal matrix with $m = 10^5$ whose eigenvalues are complex numbers scattered about the annulus $1 \le |\lambda| \le 2$.

Lecture 40. Preconditioning

The convergence of a matrix iteration depends on the properties of the matrix—the eigenvalues, the singular values, or sometimes other information. One of the developments that made it possible for these methods to take off in the 1970s and 1980s was the discovery that in many cases, the problem of interest can be transformed so that the properties of the matrix are improved drastically. This process of "preconditioning" is essential to most successful applications of iterative methods.

Preconditioners for $Ax = b$

In the abstract, the idea of preconditioning a system of equations is elementary. Suppose we wish to solve an $m \times m$ nonsingular system

$$Ax = b. \tag{40.1}$$

For any nonsingular $m \times m$ matrix M, the system

$$M^{-1}Ax = M^{-1}b \tag{40.2}$$

has the same solution. If we solve (40.2) iteratively, however, the convergence will depend on the properties of $M^{-1}A$ instead of those of A. If this *preconditioner* M is well chosen, (40.2) may be solved much more rapidly than (40.1).

For this idea to be useful, of course, it must be possible to compute the operation represented by the product $M^{-1}A$ efficiently. As usual in numerical

linear algebra, this will not mean an explicit construction of the inverse M^{-1}, but the solution of systems of equations of the form

$$My = c. \tag{40.3}$$

Two extreme cases come quickly to mind. If $M = A$, then (40.3) is the same as (40.1), so applying the preconditioner is as hard as solving the original problem, and nothing has been gained. If $M = I$, then (40.2) is the same as (40.1), so applying the preconditioner is a triviality, but it accomplishes nothing. Between these extremes lie the useful preconditioners, structured enough so that (40.3) can be solved quickly, but close enough to A in some sense that an iteration for (40.2) converges more quickly than an iteration for (40.1).

What does it mean for M to be "close enough to A?" Answering this question is the matter that has occupied our attention throughout this part of the book. If the eigenvalues of $M^{-1}A$ are close to 1 and $\|M^{-1}A - I\|_2$ is small, then any of the iterations we have discussed can be expected to converge quickly (Exercise 40.1). However, preconditioners that do not satisfy such a strong condition may also perform well. For example, the eigenvalues of $M^{-1}A$ could be clustered about a number other than 1, and there might be some outlier eigenvalues far from the others. For another example, if CGN is the iteration, it is enough for the singular values of $M^{-1}A$ to be clustered, not the eigenvalues. Detailed answers to questions of convergence rates depend, as always, on problems of polynomial approximation in the complex plane; all that changes for the analysis of preconditioners as opposed to basic iterations is that now it is the properties of $M^{-1}A$ rather than A that are of interest.

For most problems involving iterations other than CGN, fortunately, a simple rule of thumb is adequate. *A preconditioner M is good if $M^{-1}A$ is not too far from normal and its eigenvalues are clustered.*

Left, Right, and Hermitian Preconditioners

What we have described may be more precisely termed a *left preconditioner*. Another idea is to transform $Ax = b$ into $AM^{-1}y = b$, with $x = M^{-1}y$, in which case M is called a *right preconditioner*. Both left and right preconditioners are used in practice, and sometimes both are used at once. To keep the discussion simple, we shall confine our attention to the former.

If A is hermitian positive definite, then it is usual to preserve this property in preconditioning. Suppose M is also hermitian positive definite, with $M = CC^*$ for some C. Then (40.1) is equivalent to

$$[C^{-1}AC^{-*}]C^*x = C^{-1}b. \tag{40.4}$$

The matrix in brackets is hermitian positive definite, so this equation can be solved by CG or related iterations. At the same time we observe that

since $C^{-1}AC^{-*}$ is similar to $C^{-*}C^{-1}A = M^{-1}A$, it is enough to examine the eigenvalues of the nonhermitian matrix $M^{-1}A$ to investigate convergence.

Example

Figure 40.1 presents an example of a preconditioned CG iteration for a symmetric positive definite matrix. The matrix A is adapted from Exercise 36.3: it is the 1000×1000 symmetric matrix whose entries are all zero except for $a_{ij} = 0.5 + \sqrt{i}$ on the diagonal, $a_{ij} = 1$ on the sub- and superdiagonals, and $a_{ij} = 1$ on the 100th sub- and superdiagonals, i.e., for $|i-j| = 100$. The right-hand side is $b = (1, 1, \ldots, 1)^T$. As the figure shows, a straight CG iteration for this matrix converges slowly, achieving about five-digit residual reduction after forty iterations. Since the matrix is very sparse, this is an improvement over a direct method, but one would like to do better.

As it happens, we can do much better with a simple diagonal preconditioner. Take $M = \mathrm{diag}(A)$, the diagonal matrix with entries $m_{ii} = 0.5 + \sqrt{i}$. To preserve symmetry, set $C = \sqrt{M}$ and consider a new iteration preconditioned as in (40.4). The figure shows that thirty steps of the iteration now give convergence to fifteen digits.

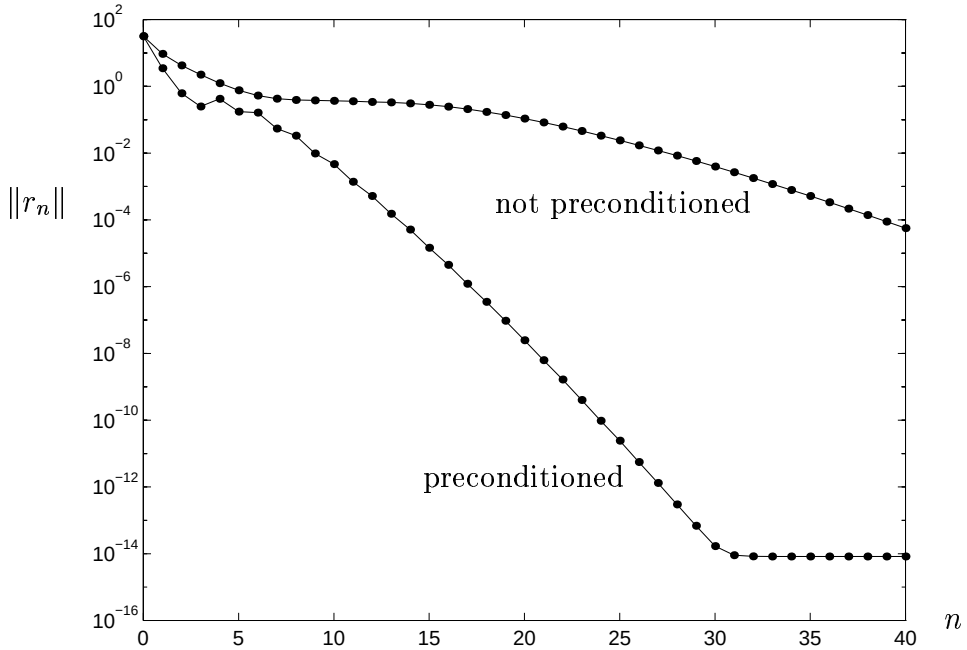


Figure 40.1. *CG and preconditioned CG convergence curves for the* 1000×1000 *sparse matrix A described in the text (the matrix of Exercise* 36.3 *plus* $0.5I$*).*

Survey of Preconditioners for $Ax = b$

The preconditioners used in practice are sometimes as simple as this one, but they are often far more complicated. Rather than consider one or two examples in detail, we shall take the opposite course and survey at a high level the wide range of preconditioning ideas that have been found useful over the years. Details can be found in the references listed in the Notes.

Diagonal scaling or Jacobi. Perhaps the most important preconditioner is the one just mentioned in the example: $M = \mathrm{diag}(A)$, provided that this matrix is nonsingular. For certain problems, this transformation alone is enough to make a slow iteration into a fast one. More generally, one may take $M = \mathrm{diag}(c)$ for a suitably chosen vector $c \in \mathbb{C}^m$. It is a hard mathematical problem to determine a vector c such that $\kappa(M^{-1}A)$ is exactly minimized, but fortunately, nothing like the exact minimum is needed in practice, and in any case, as the rule of thumb above shows, there is more to preconditioning than minimizing the condition number.

Incomplete Cholesky or LU factorization. Another star preconditioner is the one that made the idea of preconditioning famous in the 1970s. Suppose A is sparse, having just a few nonzeros per row. The difficulty with methods such as Gaussian elimination or Cholesky factorization is that these processes destroy zeros, so that if $A = R^*R$, for example, then the factor R will usually not be very sparse. However, suppose a matrix $\tilde{R}$ is computed by Cholesky-like formulas but allowed to have nonzeros only in positions where A has nonzeros, and we define $M = \tilde{R}^*\tilde{R}$. This *incomplete Cholesky* preconditioner may be highly effective for some problems; the acronym *ICCG* for incomplete Cholesky conjugate gradients is used. Similar *ILU* or *incomplete LU* preconditioners are useful in nonsymmetric cases. Numerous variants of the idea of incomplete factorization have been proposed and developed extensively.

These two examples of preconditioners are defined without reference to the origin of the underlying problem $Ax = b$. The best general advice one can give for designing preconditioners, however, is to examine that problem and take advantage of its structure. If it were simpler in a certain way, one asks, could it be solved quickly? If so, that simpler version of the problem may be an effective preconditioner. Most of our remaining examples are in this category.

Coarse-grid approximation. A discretization of a partial differential or integral equation on a fine grid may lead to a huge system of equations. The analogous discretization on a coarser grid, however, may lead to a small system that is easy to solve. If a method can be found to transfer solutions on the coarse grid to the fine grid and back again, e.g. by interpolation, then a powerful preconditioner may be obtained of the following schematic form:

$$M = \langle \text{transfer to fine grid} \rangle \circ A_{\text{coarse}} \circ \langle \text{transfer to coarse grid} \rangle. \qquad (40.5)$$

Typically a preconditioner of this kind does a good job of handling the low-

frequency components of the original problem, leaving the high frequencies to be treated by the Krylov subspace iteration. When this technique is iterated, resulting in a sequence of coarser and coarser grids, we obtain the idea of *multigrid iteration*.

Local approximation. A coarse-grid approximation takes into account some of the larger-scale structure of a problem while ignoring some of the finer structure. A kind of a converse to this idea is relevant to problems $Ax = b$ where A represents coupling between elements both near and far from one another. The elements may be physical objects such as particles, or they may be numerical objects such as the panels introduced in a boundary element discretization. In any case, it may be worth considering the operator M analogous to A but with the longer-range interactions omitted—a short-range approximation to A. In the simplest cases of this kind, M may consist simply of a few of the diagonals of A near the main diagonal, making this a generalization of the idea of a diagonal preconditioner.

Block preconditioners and domain decomposition. Throughout numerical linear algebra, most algorithms expressed in terms of the scalar entries of a matrix have analogues involving block matrices. An example is that a diagonal or Jacobi preconditioner may be generalized to block-diagonal or block-Jacobi form. This is another kind of local approximation, in that local effects within certain components are considered while connections to other components are ignored. In the past decade ideas of this kind have been widely generalized in the field of *domain decomposition*, in which solvers for certain subdomains of a problem are composed in flexible ways to form preconditioners for the global problem. These methods combine mathematical power with natural parallelizability.

Low-order discretization. Often a differential or integral equation is discretized by a higher-order method such as a fourth-order finite difference formula or a spectral method, bringing a gain in accuracy but making the discretization stencils bigger and the matrix less sparse. A lower-order approximation of the same problem, with its sparser matrix, may be an effective preconditioner. Thus, for example, one commonly encounters finite difference and finite element preconditioners for spectral discretizations.

Constant-coefficient or symmetric approximation. Special techniques, like fast Poisson solvers, are available for certain partial differential equations with constant coefficients. For a problem with variable coefficients, a constant-coefficient approximation implemented by a fast solver may make a good preconditioner. Analogously, if a differential equation is not self-adjoint but is close in some sense to a self-adjoint equation that can be solved more easily, then the latter may sometimes serve as a preconditioner.

Splitting of a multi-term operator. Many applications involve combinations of physical effects, such as the diffusion and convection that combine to make up the Navier–Stokes equations of fluid mechanics. The linear algebra result may be a matrix problem $Ax = b$ with $A = A_1 + A_2$ (or with more than two

terms, of course), often embedded in a nonlinear iteration. If A_1 or A_2 is easily invertible, it may serve as a good preconditioner.

Dimensional splitting or ADI. Another kind of splitting takes advantage of the fact that an operator such as the Laplacian in two or three dimensions is composed of analogous operators in each of the dimensions separately. This idea may form the basis of a preconditioner, and in one form goes by the name of *ADI* or *alternating direction implicit* methods.

One step of a classical iterative method. In this book we have not discussed the "classical iterations" such as Jacobi, Gauss–Seidel, SOR, or SSOR, but one or more steps of these iterations—particularly Jacobi and SSOR—often serve excellently as preconditioners. This is also one of the key ideas behind multigrid methods.

Periodic or convolution approximation. Throughout the mathematical sciences, boundary conditions are a source of analytical and computational difficulty. If only there were no boundary conditions, so that the problem were posed on a periodic domain! This idea can sometimes be the basis of a good preconditioner. In the simplest linear algebra context, it becomes the idea of preconditioning a problem involving a Toeplitz matrix A (i.e., $a_{i,j} = a_{i-j}$) by a related *circulant matrix* M ($m_{i,j} = m_{(i-j)(\bmod m)}$), which can be inverted in $O(m \log m)$ operations by a fast Fourier transform. This is a particularly well studied example in which $M^{-1}A$ may be far from the identity in norm but have highly clustered eigenvalues.

Unstable direct method. Certain numerical methods, such as Gaussian elimination without pivoting, deliver inaccurate answers because of instability. If the unstable method is fast, however, why not use it as a preconditioner? This is the "fly by wire" approach to numerical computation: solve the problem carelessly but quickly, and embed that solution in a robust control system. It is a powerful idea.

Polynomial preconditioners. Finally, we mention a technique that is different from the others in that it is essentially A^{-1} rather than A itself that is approximated by the preconditioner. A polynomial preconditioner is a matrix polynomial $M^{-1} = p(A)$ with the property that $p(A)A$ has better properties for iteration than A itself. For example, $p(A)$ might be obtained from the first few terms of the Neumann series $A^{-1} = I + (I - A) + (I - A)^2 + \cdots$, or from some other expression, often motivated by approximation theory in the complex plane. Implementation is easy, based on the same "black box" used for the Krylov subspace iteration itself, and the coefficients of the preconditioner may sometimes be determined adaptively.

Preconditioners for Eigenvalue Problems

Though the idea has been developed more recently and is not yet as famous, preconditioners can be effective for eigenvalue problems as well as systems of equations. Some of the best-known techniques in this area are *polynomial*

acceleration, analogous to the polynomial preconditioning just described for systems of equations, *shift-and-invert Arnoldi* or the related *rational Krylov iteration*, which employ rational functions of A instead of polynomials, and the *Davidson* and *Jacobi–Davidson* methods, based on a kind of diagonal preconditioner. For example, shift-and-invert and rational Krylov methods are based on the fact that if $r(z)$ is a rational function and $\{\lambda_j\}$ are the eigenvalues of A, then the eigenvalues of $r(A)$ are $\{r(\lambda_j)\}$. If $r(A)$ can be computed with reasonable speed and its eigenvalues are better distributed for iteration than those of A, this may be a route to fast calculation of eigenvalues.

A Closing Note

In ending this book with the subject of preconditioners, we find ourselves at the philosophical center of the scientific computing of the future. The traditional view of computer scientists is that a computational problem is finite: after a short or long calculation, one obtains the solution exactly. Over the years, however, this view has come to be appropriate to fewer and fewer problems. The best methods for large-scale computational problems are usually approximate ones, methods that obtain a satisfactorily accurate solution in a short time rather than an exact one in a much longer or infinite time. Numerical analysis is indeed a branch of analysis, primarily, not algebra—even when the problems to be solved are from linear algebra. Further speculations on this phenomenon are presented in the Appendix.

Nothing will be more central to computational science in the next century than the art of transforming a problem that appears intractable into another whose solution can be approximated rapidly. For Krylov subspace matrix iterations, this is preconditioning. For the great range of computational problems, both continuous and discrete, we can only guess where this idea will take us.

Exercises

40.1. Suppose $A = M - N$, where M is nonsingular. Suppose $\|I - M^{-1}N\|_2 \leq 1/2$, and M is used as a preconditioner as in (40.2).

(a) Show that if GMRES is applied to this preconditioned problem, then the residual norm is guaranteed to be six orders of magnitude smaller, or better, after twenty steps.

(b) How many steps of CGN are needed for the same guarantee?

40.2. Show that if a matrix A and a preconditioner M are hermitian positive definite, then the same CG convergence rate is obtained whether M is used as a left preconditioner or a right preconditioner. Explain why this result does not hold for nonhermitian matrices and iterations such as GMRES, CGN, or BCG.

Appendix. The Definition of Numerical Analysis

*by Lloyd N. Trefethen**

What is numerical analysis? I believe that this is more than a philosophical question. A certain wrong answer has taken hold among both outsiders to the field and insiders, distorting the image of a subject at the heart of the mathematical sciences.

Here is the wrong answer:

Numerical analysis is the study of rounding errors. (D1)

The reader will agree that it would be hard to devise a more uninviting description of a field. Rounding errors are inevitable, yes, but they are complicated and tedious and—*not fundamental*. If (D1) is a common perception, it is hardly surprising that numerical analysis is widely regarded as an unglamorous subject. In fact, mathematicians, physicists, and computer scientists have all tended to hold numerical analysis in low esteem for many years—a most unusual consensus.

*This essay is reprinted from the November 1992 issue of *SIAM News*. It was reprinted previously in the March/April 1993 issue of the *Bulletin of the Institute of Mathematics and Its Applications*.